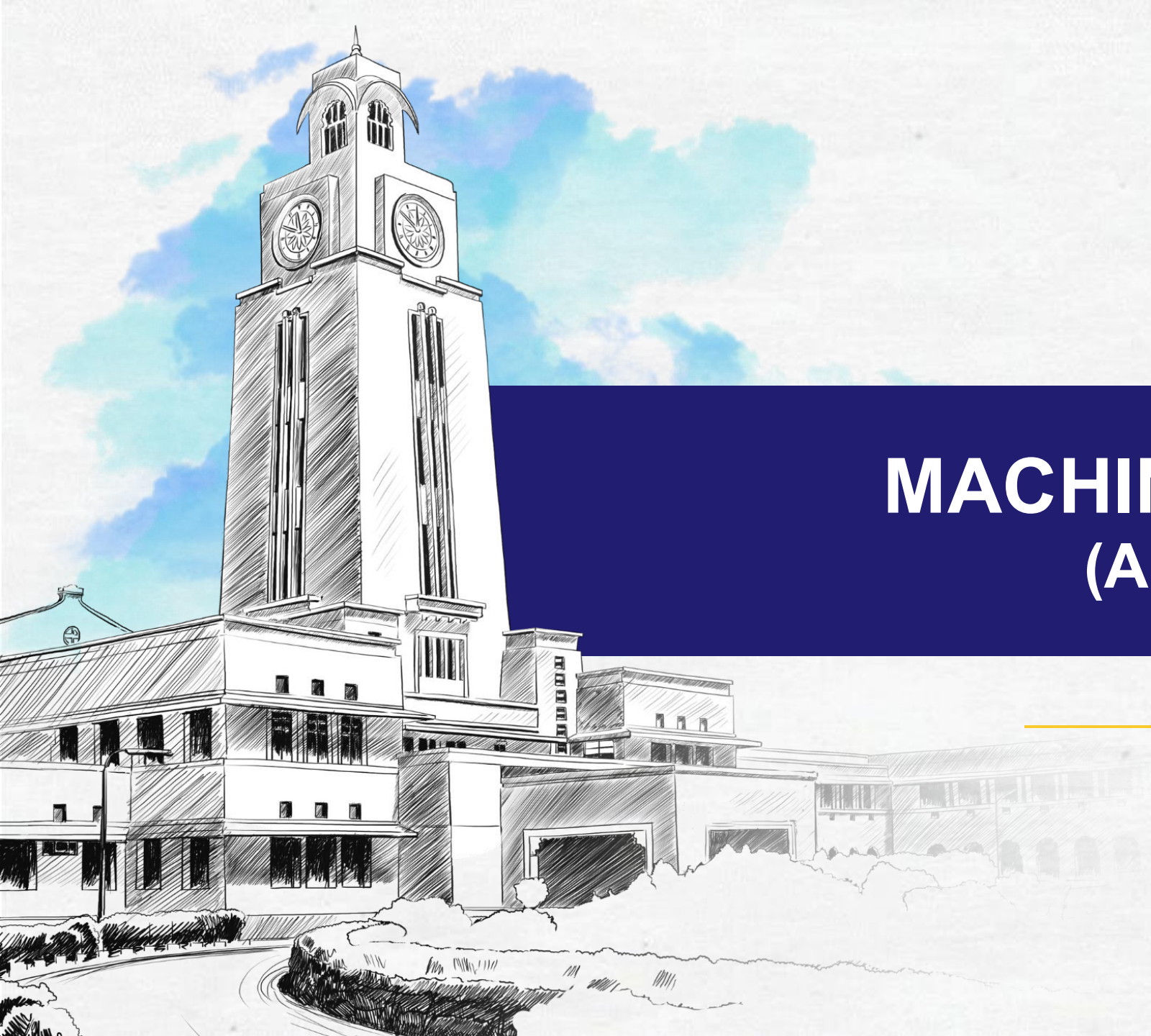




BITS Pilani
Pilani | Dubai | Goa | Hyderabad

MACHINE LEARNING (AIMLZG565)



Session 3&4
(16th August, 2026)

Course Plan

M1	Introduction
M2	Machine learning Workflow
M3	Linear Models for Regression
M4	Linear Models for Classification
M5	Decision Tree
M6	Instance Based Learning
M7	Support Vector Machine
M8	Bayesian Learning
M9	Ensemble Learning
M10	Unsupervised Learning
M11	Machine Learning Model Evaluation/Comparison

Agenda

- Linear Model for Regression
- Direct solution vs Iterative Method
- Gradient Descent
- Linear Basis Function
- Notion of Bias vs Variance





Linear Regression

Inductive Learning Hypothesis : Interpretation

- Target Concept
- Discrete : $f(x) \in \{\text{Yes, No, Maybe}\}$ Classification
- Continuous : $f(x) \in [20-100]$ Regression
- Probability Estimation : $f(x) \in [0-1]$

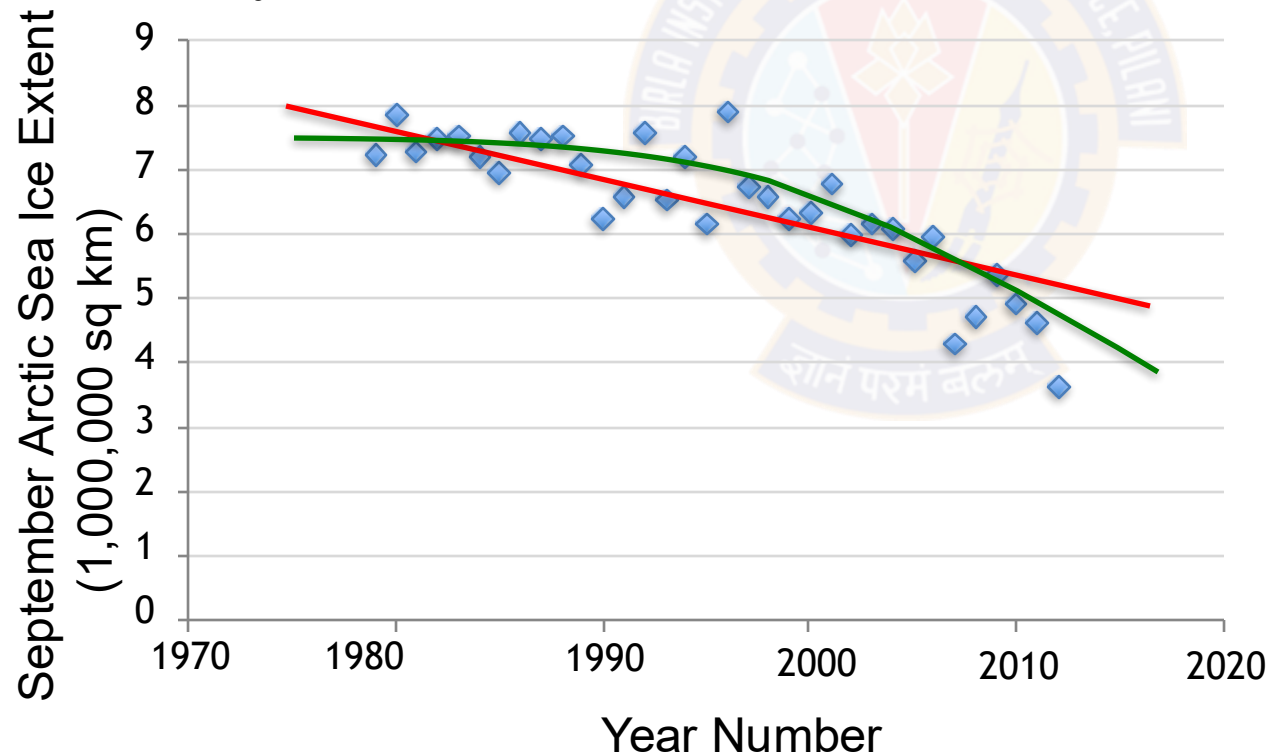
Sky	AirTemp	Altitude	Wind	Water	Forecast	Humidity
Sunny	Warm	Normal	Strong	Warm	Same	60
Sunny	Warm	High	Strong	Warm	Same	75
Rainy	Cold	High	Strong	Warm	Change	70
Sunny	Warm	High	Strong	Cool	Change	45

Formal Representation: Interpretation

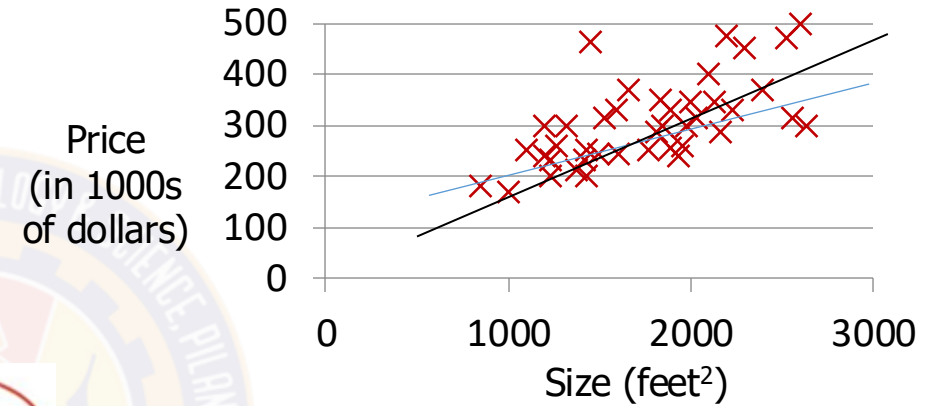
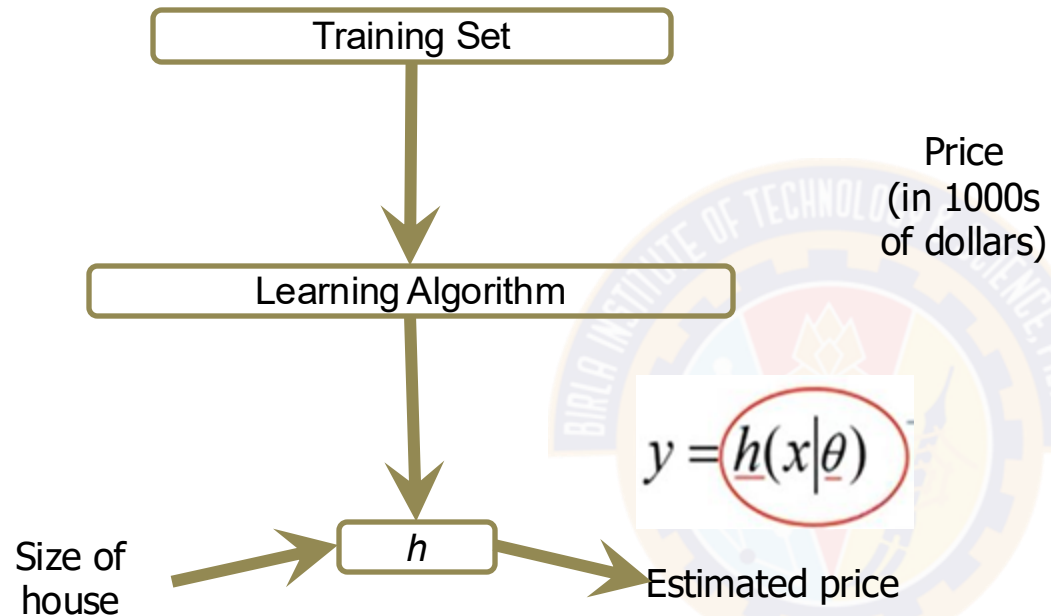
Supervised Learning: Regression

GOAL : Previously unseen records should be assigned a value as accurately as possible.

- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is real-valued



Machine Learning Process : Interpretation



Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

θ_i 's : Parameters

$h(.)$: Model

How to choose θ_i 's ?

Types of Regression Models

Simple Linear regression model

- (Education) $x \longrightarrow y$ (Income)

Multiple Linear regression model

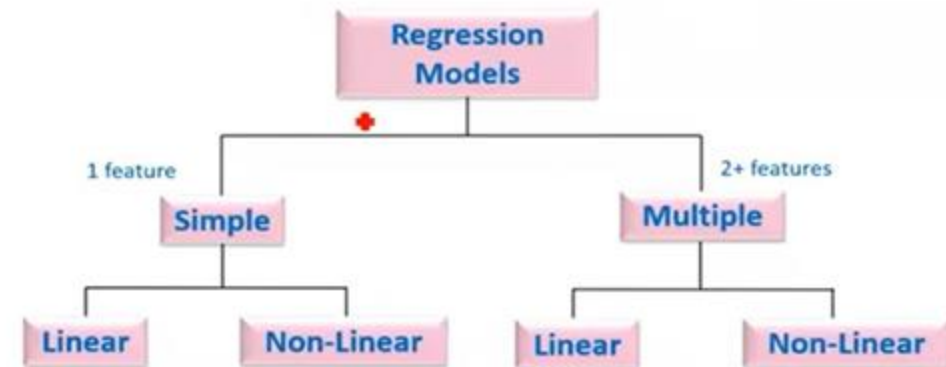
$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x,$$

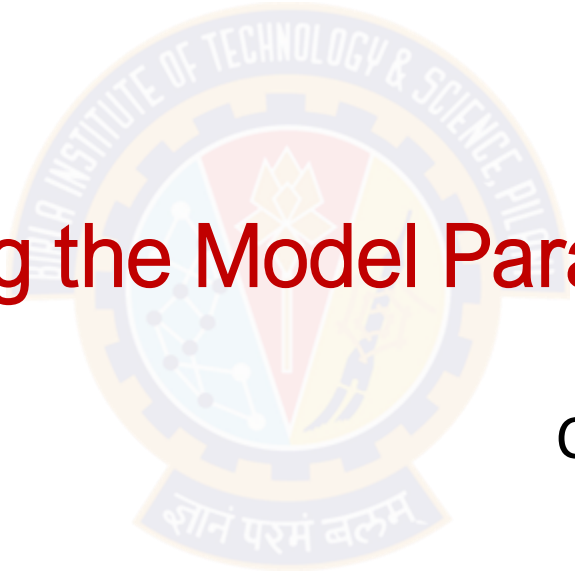
- (Education) x_1
 - (Soft Skills) x_2
 - (Experience) x_3
 - (Age) x_4
- y (Income)

- Hypothesis:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \sum_{j=0}^d \theta_j x_j$$

Assume $x_0 = 1$





Learning the Model Parameters

Closed Form Solution Approach

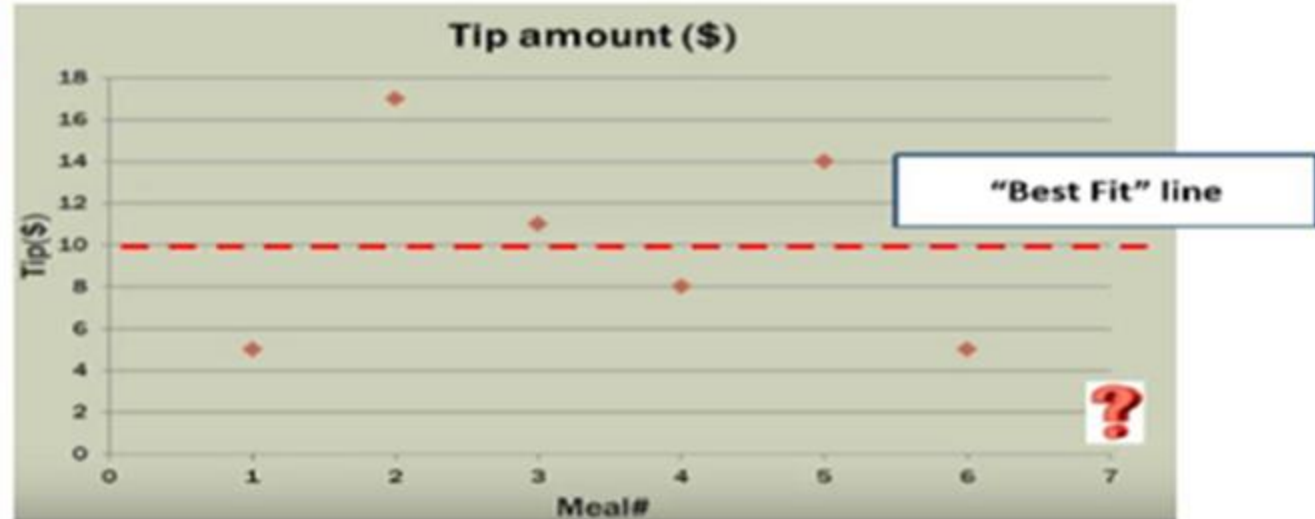
Gradient Descent Approach

How to choose θ_i 's ?

Notion of Cost Function

Meal #	Tip amount (\$)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = \$10

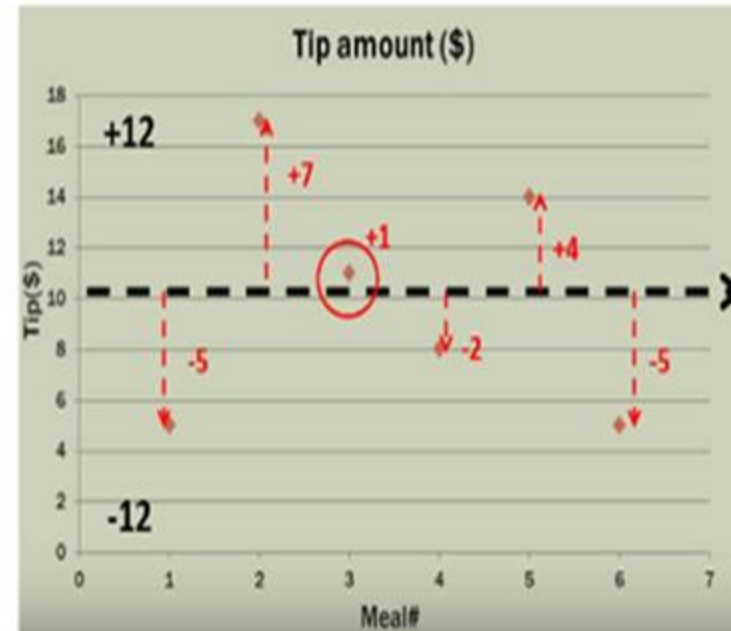
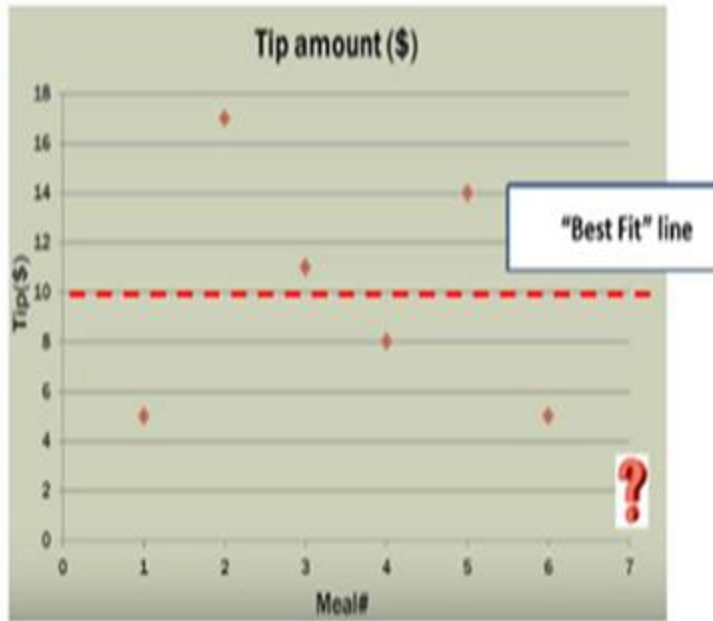


$$y = \theta_0$$

Notion of Cost Function

Meal #	Tip amount (\$)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = \$10



RESIDUALS (error): Distance from best fit line and the actual values

Meal#	Residual	Residual ²
1	-5	25
2	+7	49
3	+1	1
4	-2	4
5	+4	16
6	-5	25

Sum of Squared Errors (SSE) = 120

Compute θ_1 (slope):

$$\theta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Meal	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	34	5	-40	-5	200	1600
2	108	17	34	7	238	1156
3	64	11	-10	1	-10	100
4	88	8	14	-2	-28	196
5	99	14	25	4	100	625
6	51	5	-23	-5	115	529

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = 200 + 238 - 10 - 28 + 100 + 115 = 615$$

$$\sum (x_i - \bar{x})^2 = 1600 + 1156 + 100 + 196 + 625 + 529 = 4206$$

$$\theta_1 = \frac{615}{4206} \approx 0.1462$$



Compute θ_0 (intercept):

$$\theta_0 = \bar{y} - \theta_1 \bar{x} = 10 - (0.1462 \times 74)$$

$$\theta_0 = 10 - 10.8188 = -0.8188$$

Notion of Cost Function

Meal #	Tip amount (\$)
1	5.00
2	17.00
3	11.00
4	8.00
5	14.00
6	5.00

mean = \$10

Meal	Total bill (\$)	Tip amount (\$)	$\hat{y}_i = 0.1462x - 0.8188$	$\hat{y}_i$ (predicted tip amount)
	x	y		
1	34	5	$\hat{y}_1 = 0.1462(34) - 0.8188$	4.1505
2	108	17	$\hat{y}_2 = 0.1462(108) - 0.8188$	14.9693
3	64	11	$\hat{y}_3 = 0.1462(64) - 0.8188$	8.5365
4	88	8	$\hat{y}_4 = 0.1462(88) - 0.8188$	12.0453
5	99	14	$\hat{y}_5 = 0.1462(99) - 0.8188$	13.6535
6	51	5	$\hat{y}_6 = 0.1462(51) - 0.8188$	6.6359
	$\bar{x} = 74$	$\bar{y} = 10$	Observed vs. Predicted	

$$y = \theta_0 + \theta_1 x$$

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

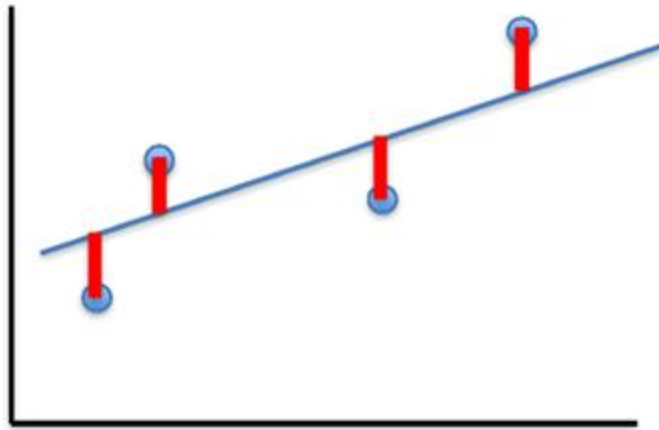
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Linear Regression : Least Squares ✓

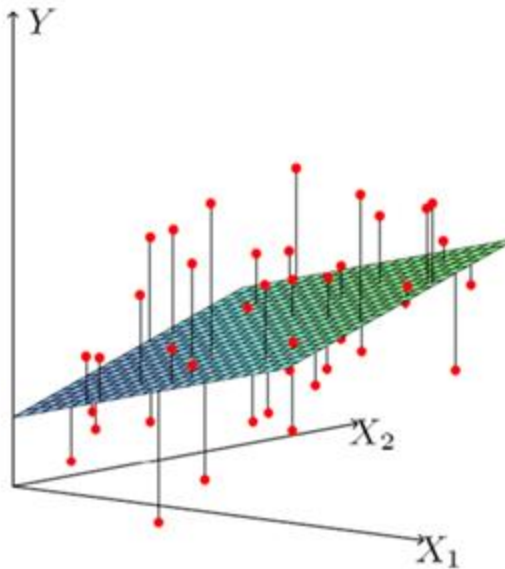
- Cost Function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

- Fit by solving $\min_{\theta} J(\theta)$



2D



- Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

- Parameters

$$\theta_0 \text{ and } \theta_1$$

- Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

- Goal

$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$$

Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

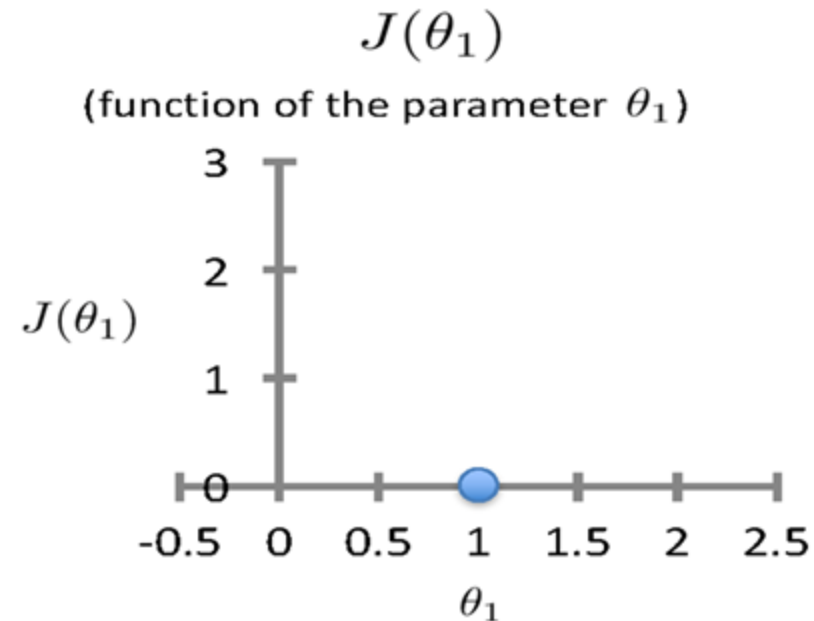
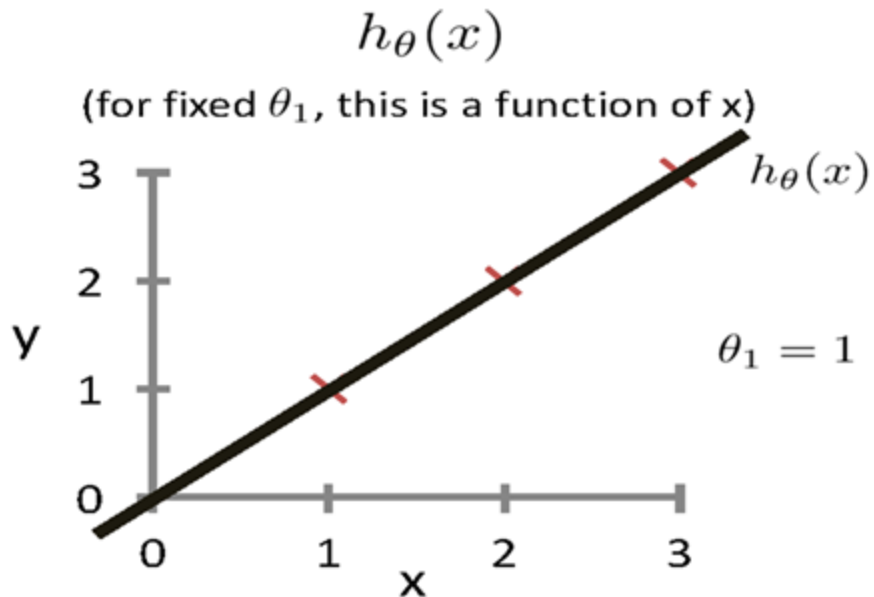
For insight on $J()$, let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$



Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

For insight on $J()$, let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$



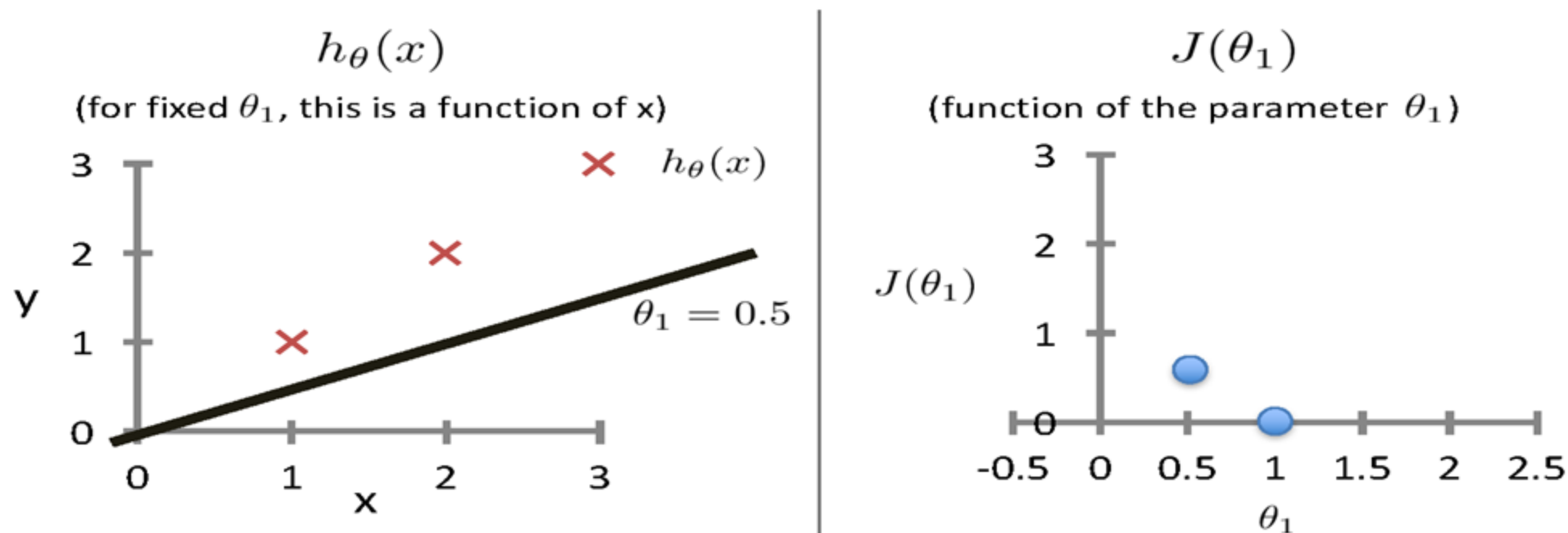
Based on example

Source Credit : by Andrew Ng

Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2$$

For insight on $J()$, let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$



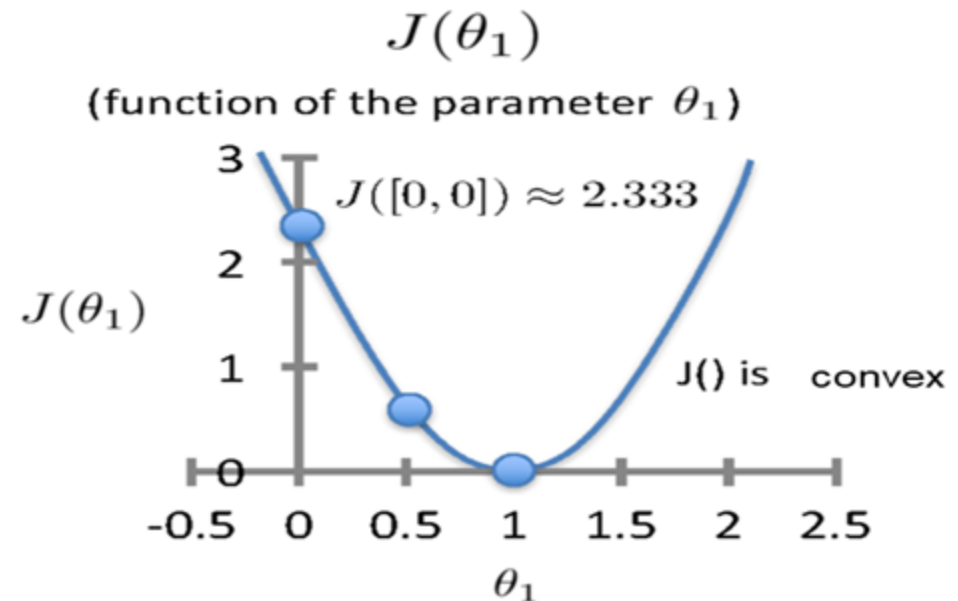
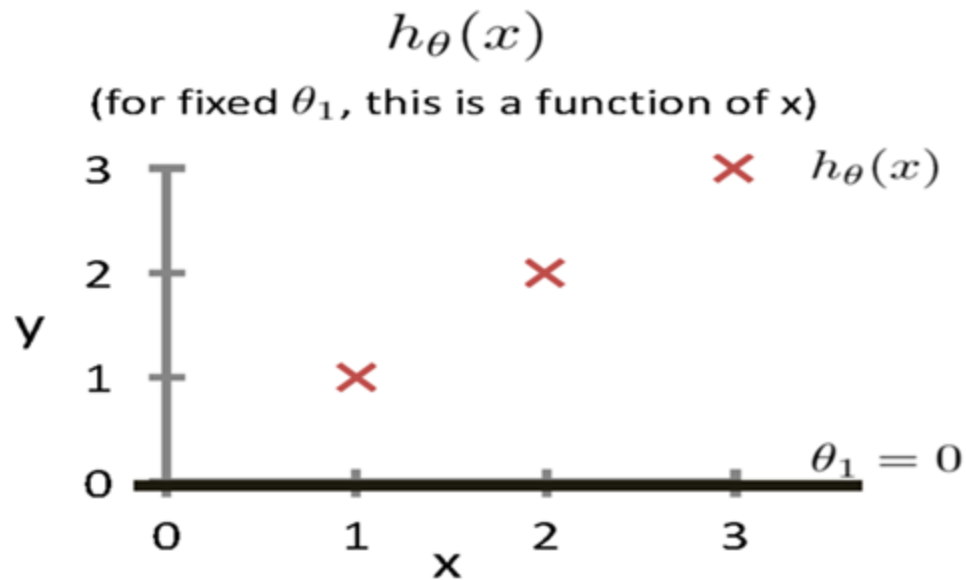
Based on example
by Andrew Ng

$$J([0, 0.5]) = \frac{1}{2 \times 3} [(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2] \approx 0.58$$

Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

For insight on $J()$, let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$

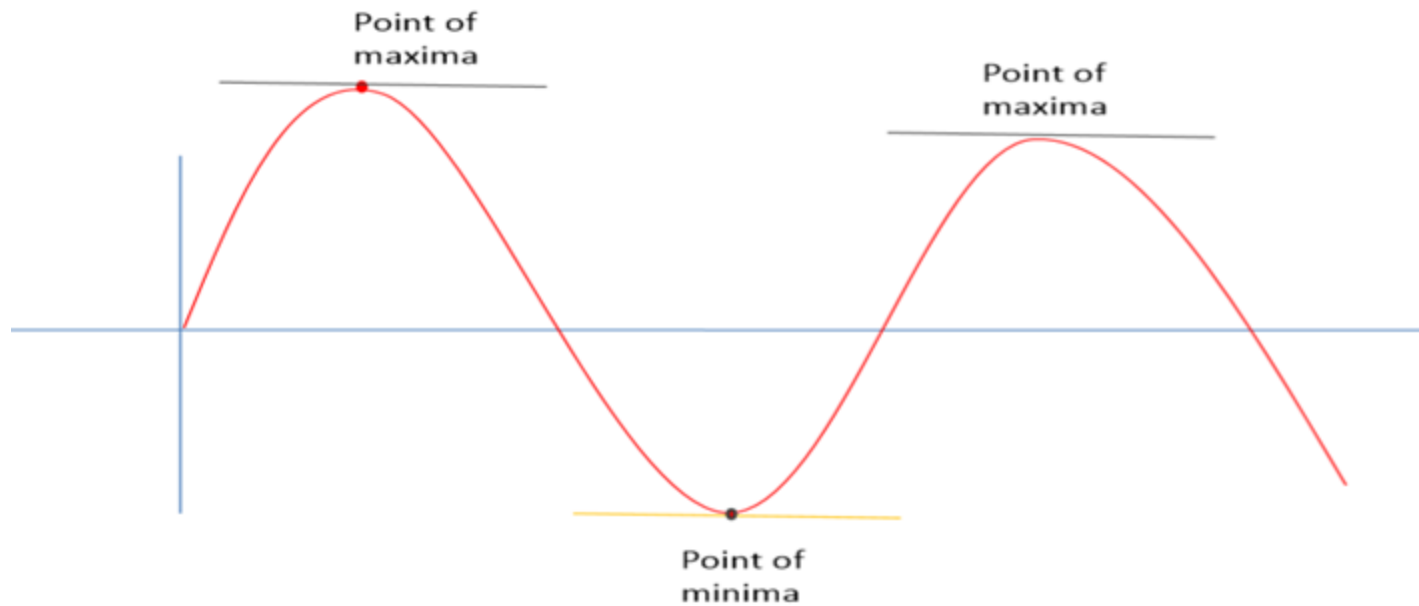


Based on example

Source Credit : by Andrew Ng

Notion of Maxima and Minima of a function - In General

- For maxima and minima $m = \frac{dy}{dx} = \tan \theta = 0$
- $\frac{dy}{dx} = 0$ means tangent is parallel to X-axis.



Notion of Maxima and Minima of a function - In Machine Learning

$\text{CarPrice} = 1 + 0.05\text{Mileage}$

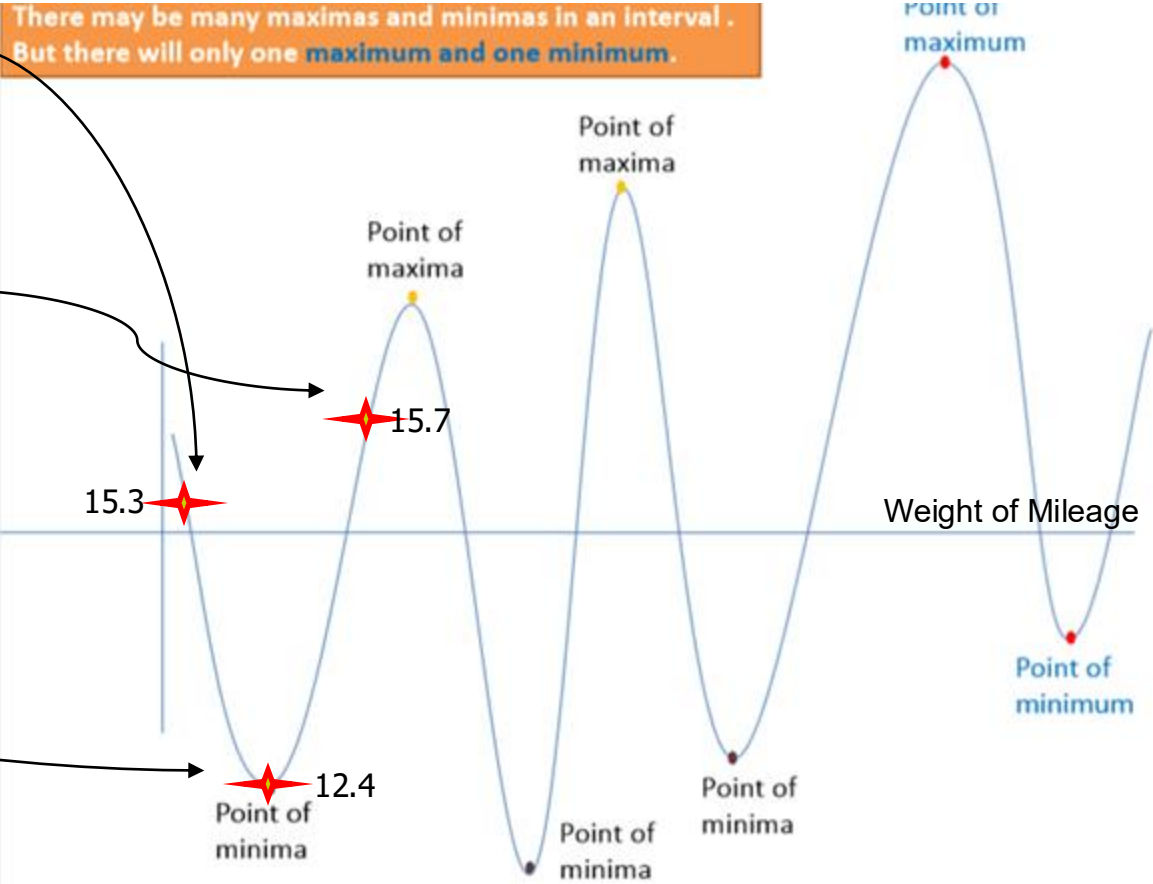
$\text{CarPrice} = 1 + 0.25\text{Mileage}$

$\text{CarPrice} = 1 + 0.75\text{Mileage}$

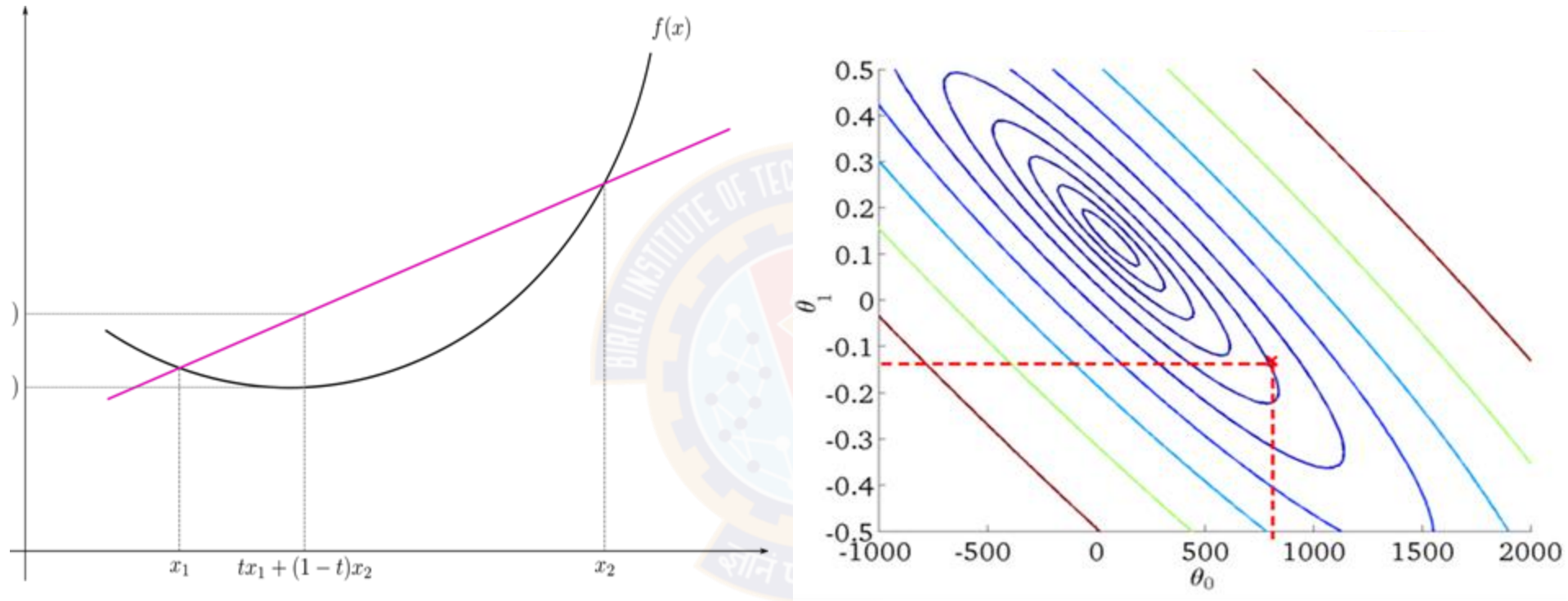
Mileage (in kmpl)	Car Price (in cr)
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51

Sum of absolute Error
/ Training Error

There may be many maximas and minimas in an interval .
But there will only one **maximum** and one **minimum**.



Convex function: Multivariate View



Real-valued function defined on an n -dimensional interval is called **convex** if the line segment between any two points on the graph of the function lies above or on the graph

Closed Form Solution



Vectorization

- Benefits of vectorization
 - More compact equations
 - Faster code (using optimized matrix libraries)

- Consider our model:

$$h(\mathbf{x}) = \sum_{j=0}^d \theta_j x_j$$

- Let

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \quad \mathbf{x}^\top = \begin{bmatrix} 1 & x_1 & \dots & x_d \end{bmatrix}$$

- Can write the model in vectorized form as $h(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$

$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad x^T = \begin{bmatrix} 1 & \text{No.of.Years Experience} \end{bmatrix}$$

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

Vectorization

- Consider our model for n instances:

$$h(x^{(i)}) = \sum_{j=0}^d \theta_j x_j^{(i)}$$

- Let

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \quad X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \quad \text{Let: } y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$\mathbb{R}^{(d+1) \times 1}$ $\mathbb{R}^{n \times (d+1)}$

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

- Can write the model in vectorized form as $h_{\theta}(x) = X\theta$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Vectorization

- For the linear regression cost function:

$$\begin{aligned}
 J(\theta) &= \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 \\
 &= \frac{1}{2n} \sum_{i=1}^n \left(\theta^T x^{(i)} - y^{(i)} \right)^2 \\
 &= \frac{1}{2n} (\underbrace{X\theta}_{\mathbb{R}^{1 \times n}} - \underbrace{y}_{\mathbb{R}^{n \times 1}})^T (\underbrace{X\theta}_{\mathbb{R}^{n \times 1}} - \underbrace{y}_{\mathbb{R}^{n \times 1}})
 \end{aligned}$$

Annotations for the matrix dimensions in the final equation:

- $\mathbb{R}^{n \times (d+1)}$ points to X
- $\mathbb{R}^{(d+1) \times 1}$ points to θ
- $\mathbb{R}^{1 \times n}$ points to $X\theta$
- $\mathbb{R}^{n \times 1}$ points to y

Let:

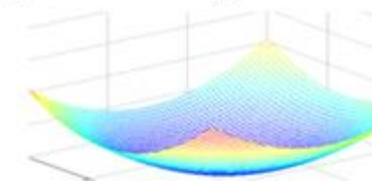
$$y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

$$\theta^T = \begin{bmatrix} \theta_0 & \theta_1 \end{bmatrix} \quad X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Vectorization

- Instead of using GD, solve for optimal θ analytically
 - Notice that the solution is when $\frac{\partial}{\partial \theta} J(\theta) = 0$



- Derivation:

$$\begin{aligned} \mathcal{J}(\theta) &= \frac{1}{2n} (\mathbf{X}\theta - \mathbf{y})^\top (\mathbf{X}\theta - \mathbf{y}) \\ &\propto \theta^\top \mathbf{X}^\top \mathbf{X} \theta - \boxed{\mathbf{y}^\top \mathbf{X} \theta} - \boxed{\theta^\top \mathbf{X}^\top \mathbf{y}} + \mathbf{y}^\top \mathbf{y} \\ &\propto \theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\theta^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y} \end{aligned}$$

Diagram: A blue arrow points from the 1×1 label to the two boxed terms $\mathbf{y}^\top \mathbf{X} \theta$ and $\theta^\top \mathbf{X}^\top \mathbf{y}$, indicating they are scalars.

Take derivative and set equal to 0, then solve for θ :

$$\frac{\partial}{\partial \theta} (\theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\theta^\top \mathbf{X}^\top \mathbf{y} + \cancel{\mathbf{y}^\top \mathbf{y}}) = 0$$

$$(\mathbf{X}^\top \mathbf{X})\theta - \mathbf{X}^\top \mathbf{y} = 0$$

$$(\mathbf{X}^\top \mathbf{X})\theta = \mathbf{X}^\top \mathbf{y}$$

Closed Form Solution:

$$\theta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Vectorization

- Can obtain θ by simply plugging X and y into

$$\theta = (X^T X)^{-1} X^T y$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

- If $X^T X$ is not invertible (i.e., singular), may need to:
 - Use pseudo-inverse instead of the inverse
 - In python, `numpy.linalg.pinv(a)`
 - Remove redundant (not linearly independent) features
 - Remove extra features to ensure that $d \leq n$

Fit the Linear Regression Model :

Using Closed Form – Problem Type 1

X No.of.Years of Experience (in Years)	Y Salary Of the Employee (in Lakhs)
1	2
2	3
3	4
4	5
5	6

$$\left(X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * X \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix} \right) - 1 * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\left(\begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix} \right) - 1 * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\left(\begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix} \right) * X^T \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Can obtain θ by simply plugging X and y into

$$\theta = (X^T X)^{-1} X^T y$$

$$\begin{bmatrix} 0.8 & 0.5 & 0.2 & -0.1 & -0.4 \\ -0.2 & -0.1 & 0 & 0.1 & -0.2 \end{bmatrix} * y \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\theta = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\theta^T = \begin{bmatrix} \theta_0 & \theta_1 \end{bmatrix} X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \end{bmatrix} y = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

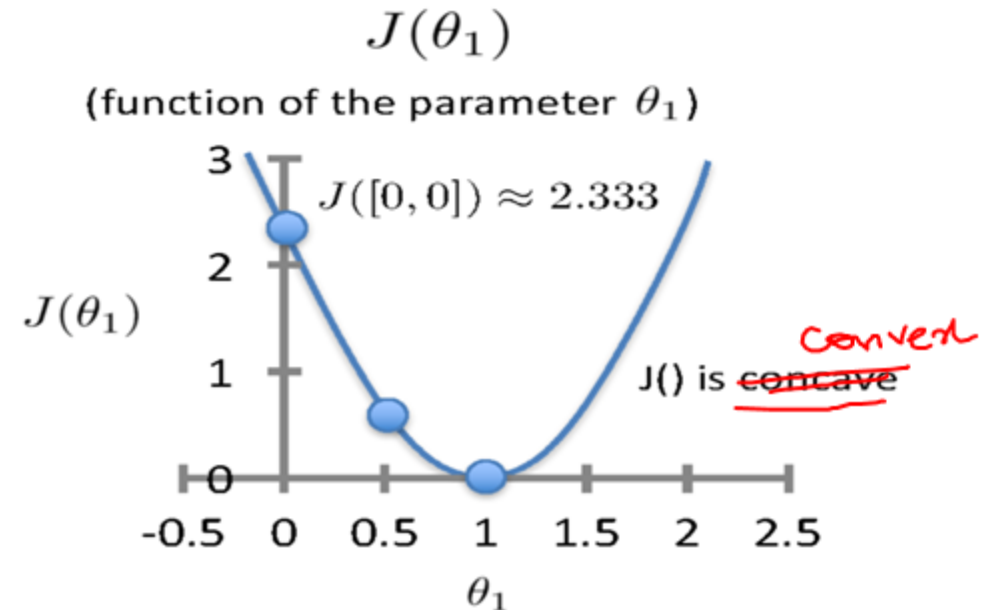
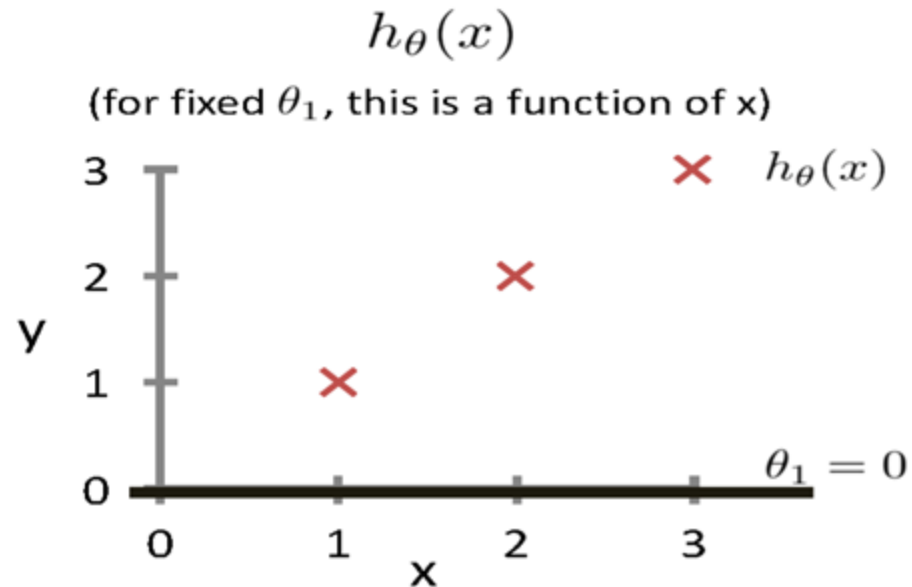
Gradient Descent Approach



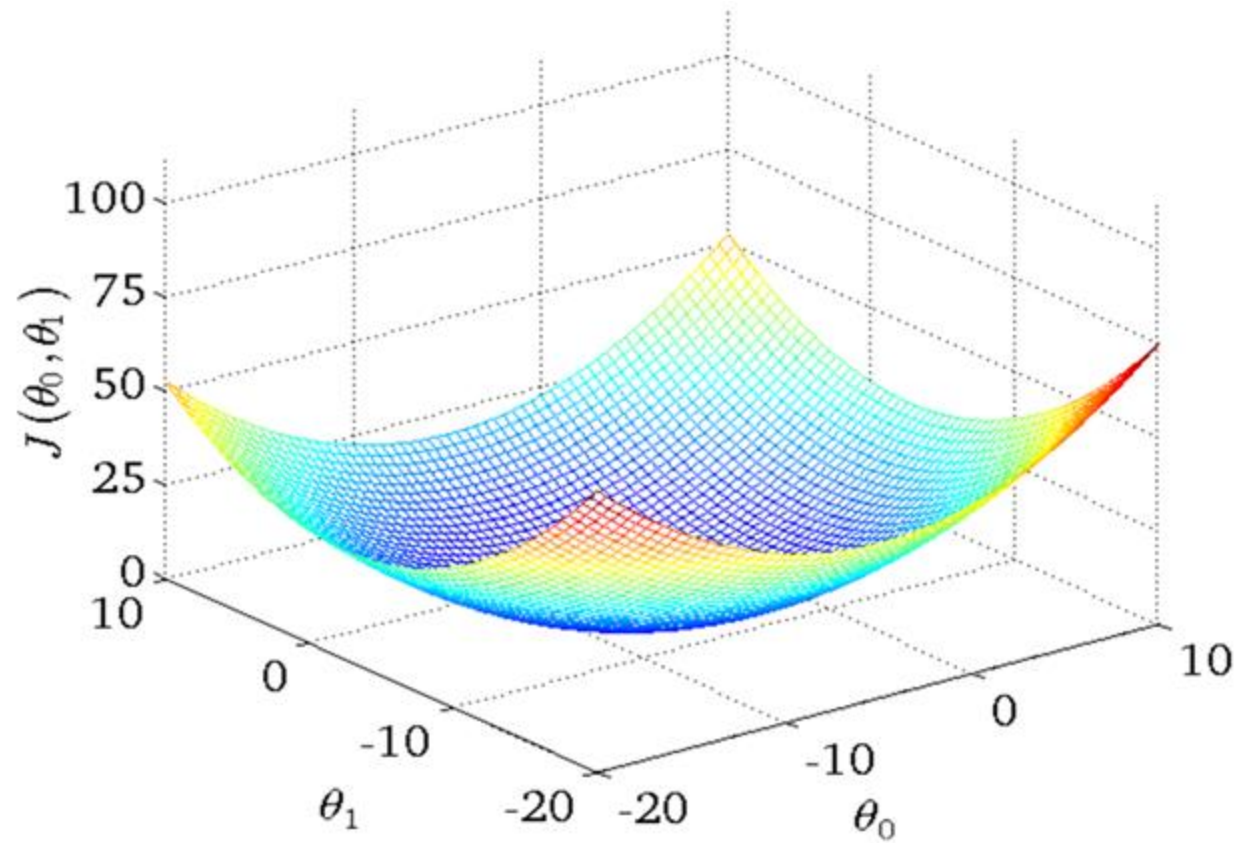
Intuition Behind Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

For insight on $J()$, let's assume $x \in \mathbb{R}$ so $\boldsymbol{\theta} = [\theta_0, \theta_1]$



Intuition Behind Cost Function

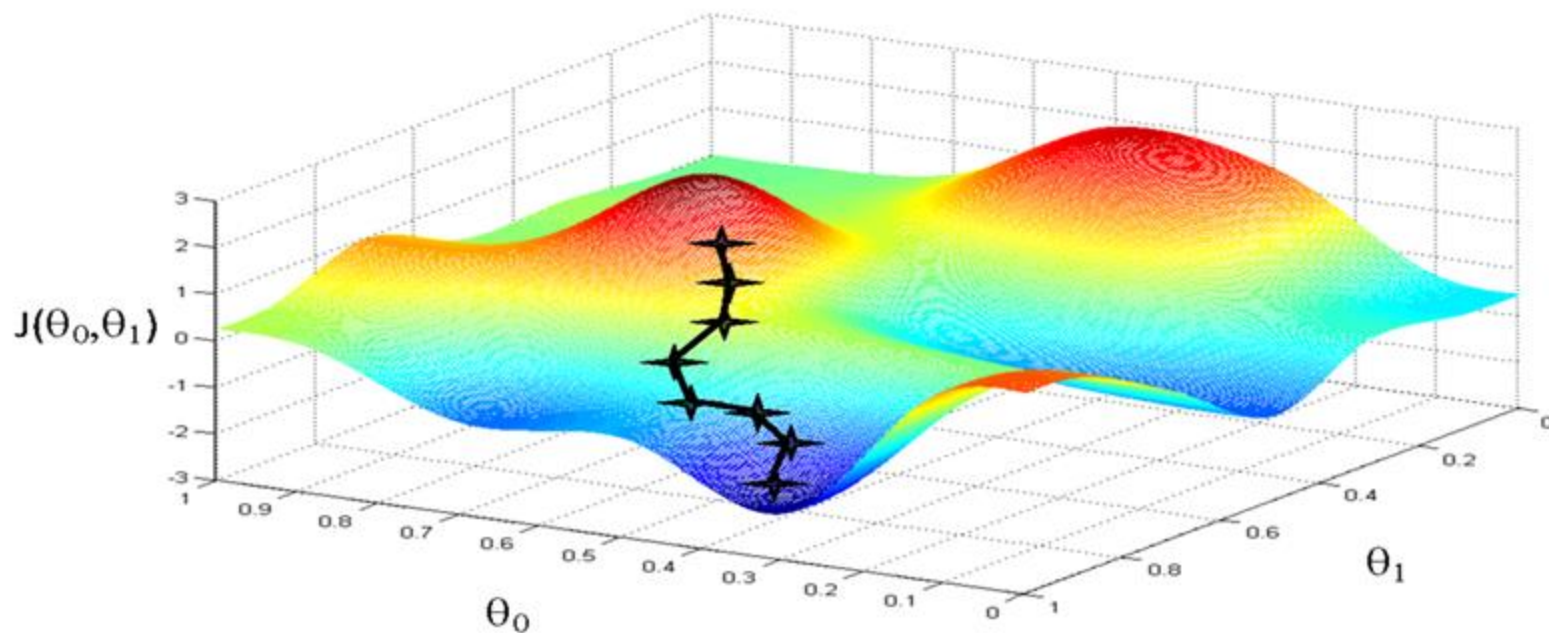


Based on example

Source Credit : by Andrew Ng

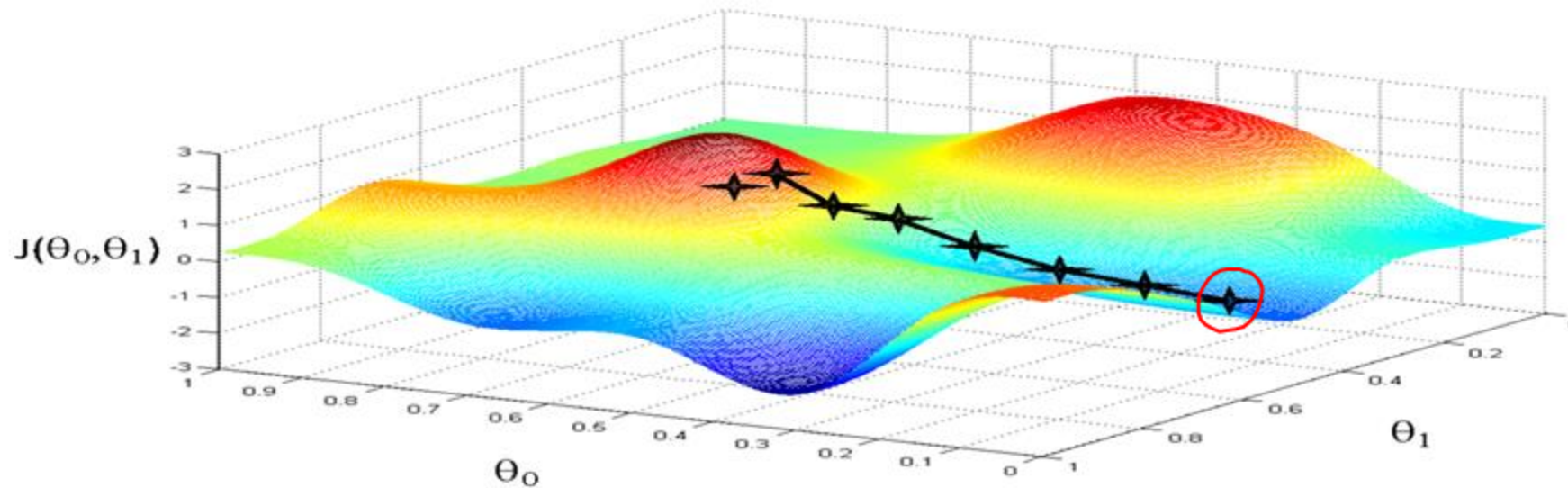
Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$



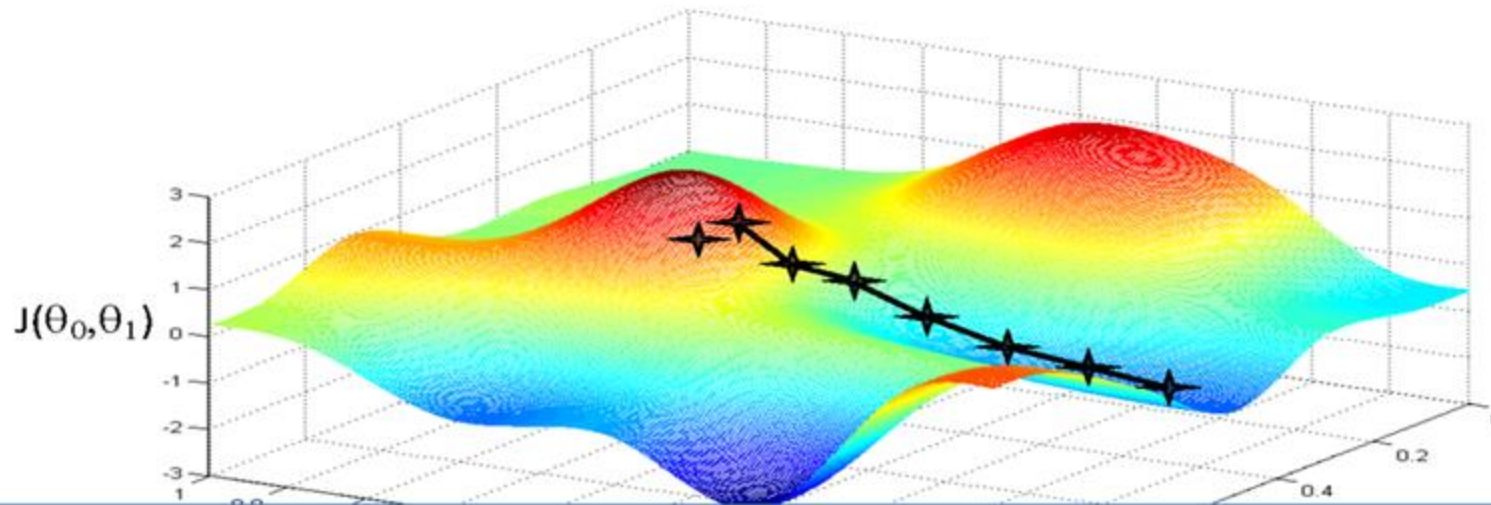
Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$



Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$



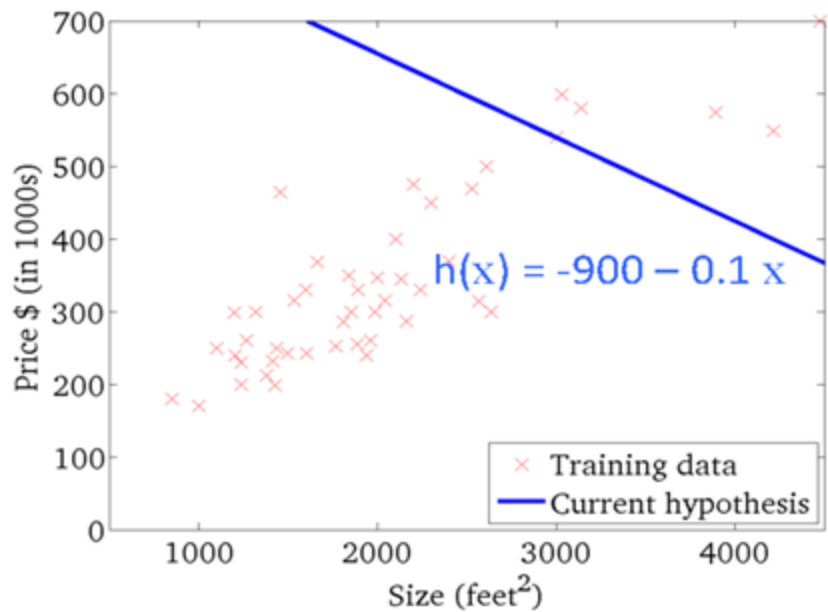
Since the least squares objective function is convex (concave), we don't need to worry about local minima

Gradient Descent

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

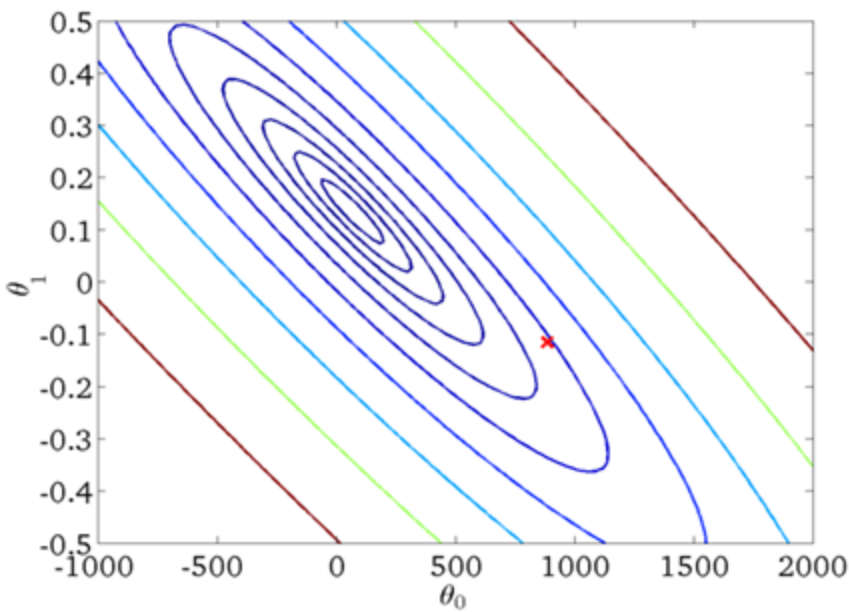
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

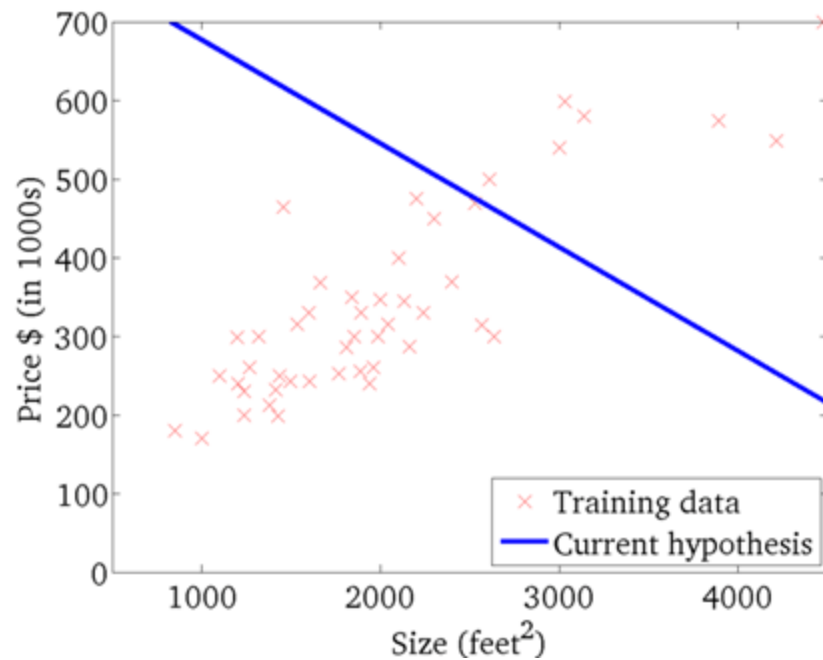
(function of the parameters θ_0, θ_1)



Gradient Descent

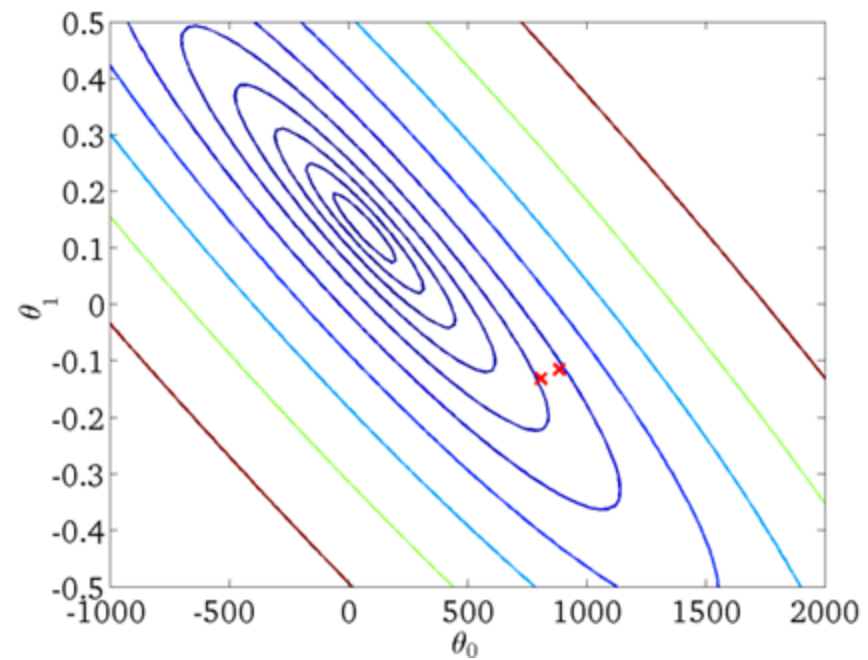
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

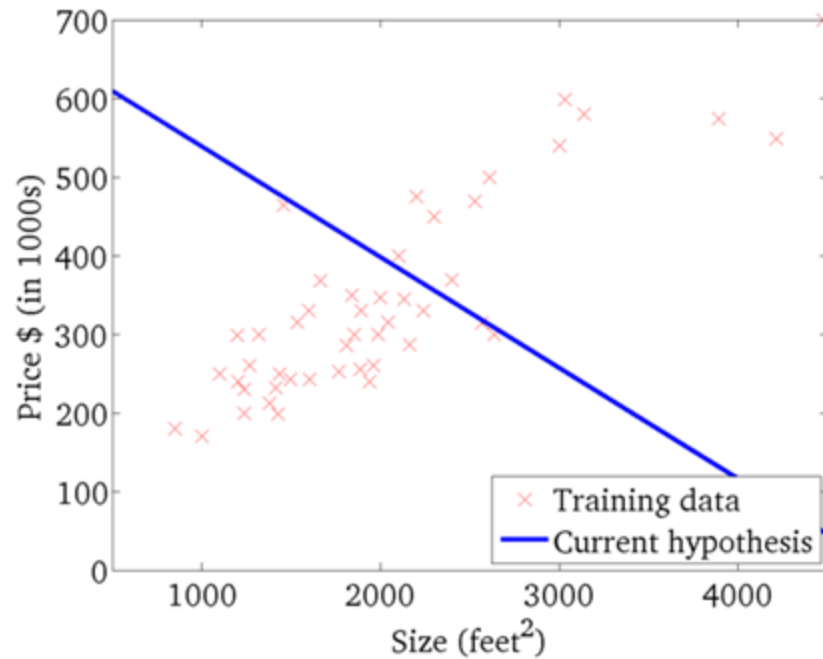
(function of the parameters θ_0, θ_1)



Gradient Descent

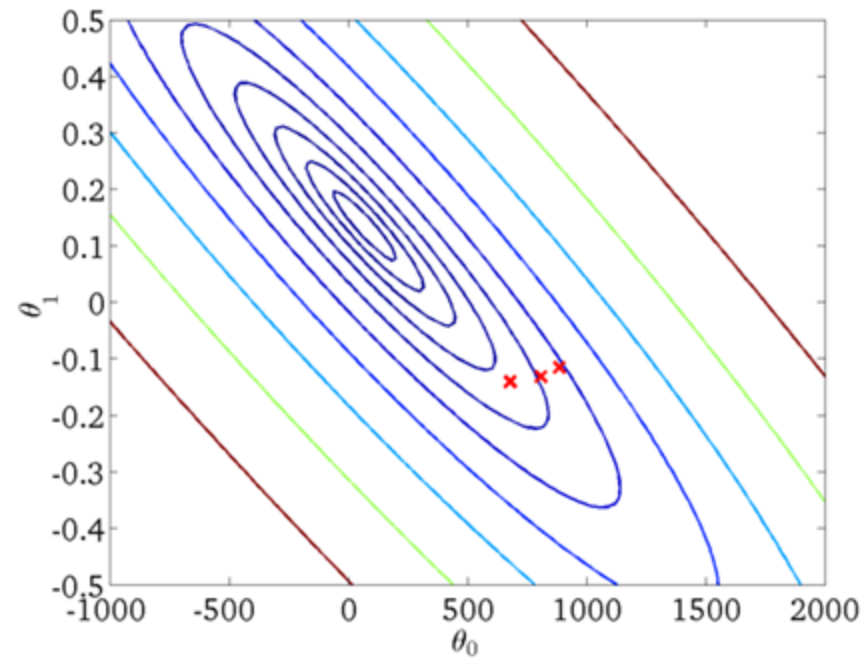
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

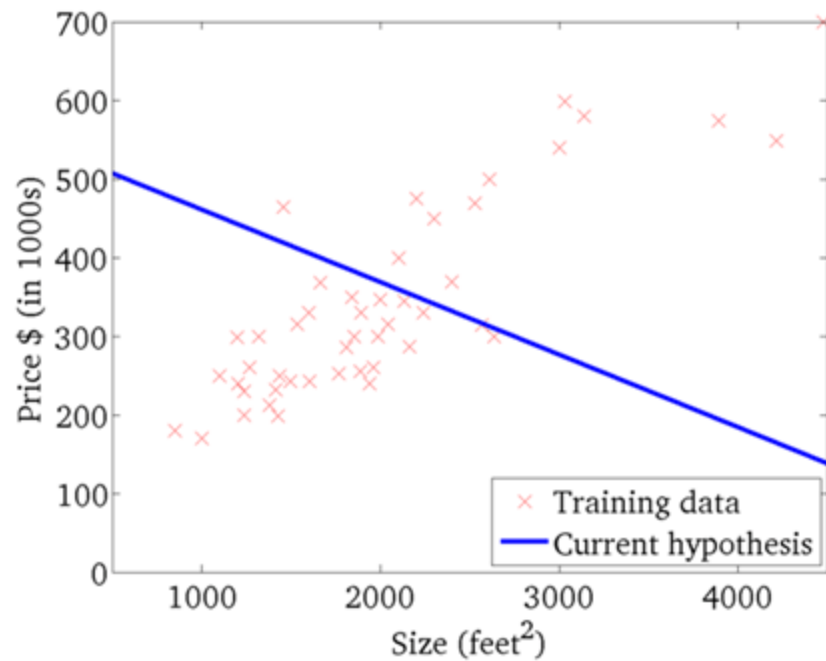
(function of the parameters θ_0, θ_1)



Gradient Descent

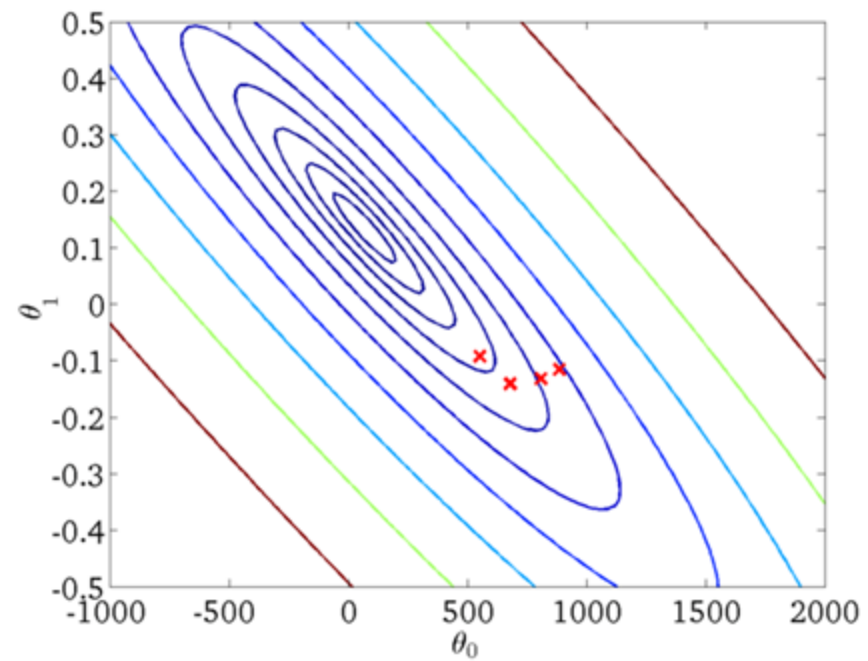
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

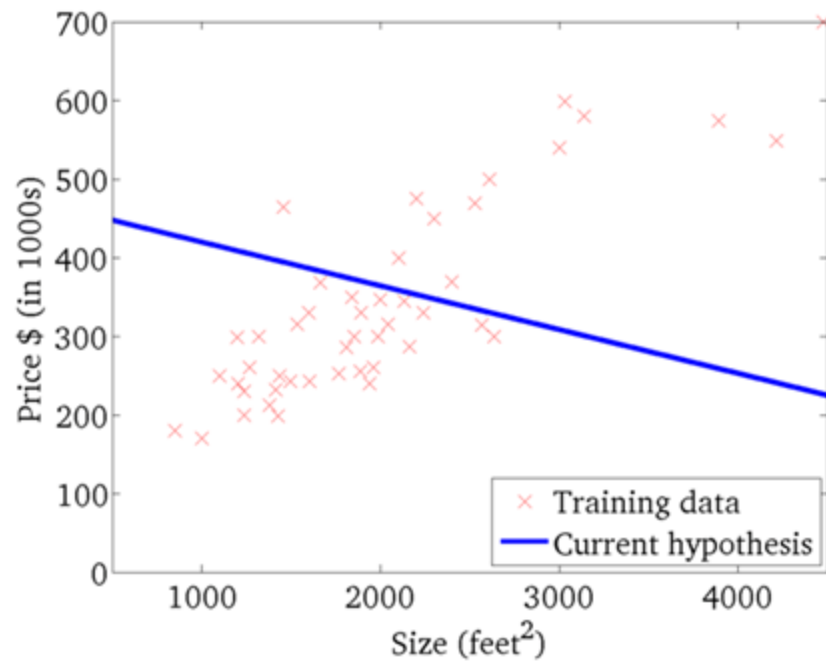
(function of the parameters θ_0, θ_1)



Gradient Descent

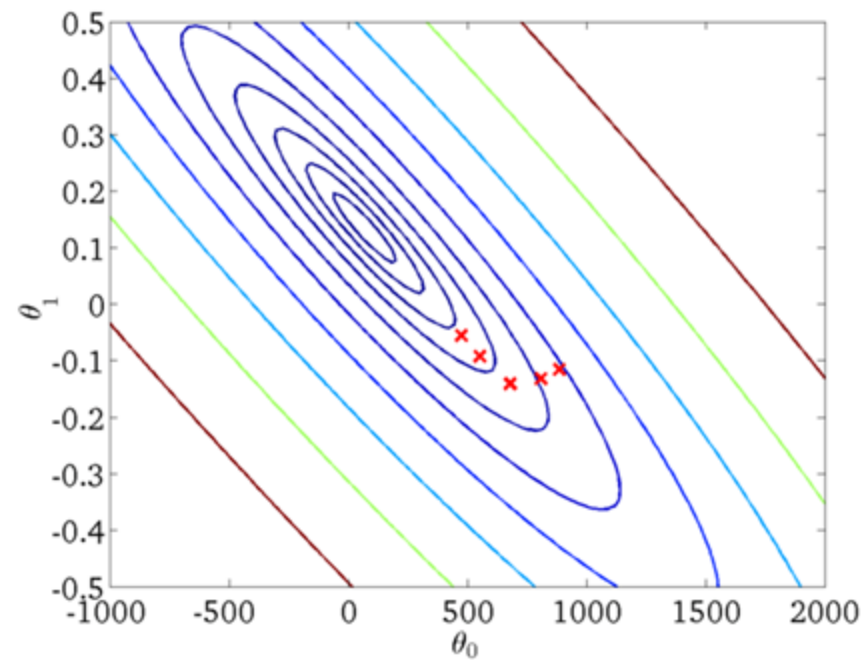
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

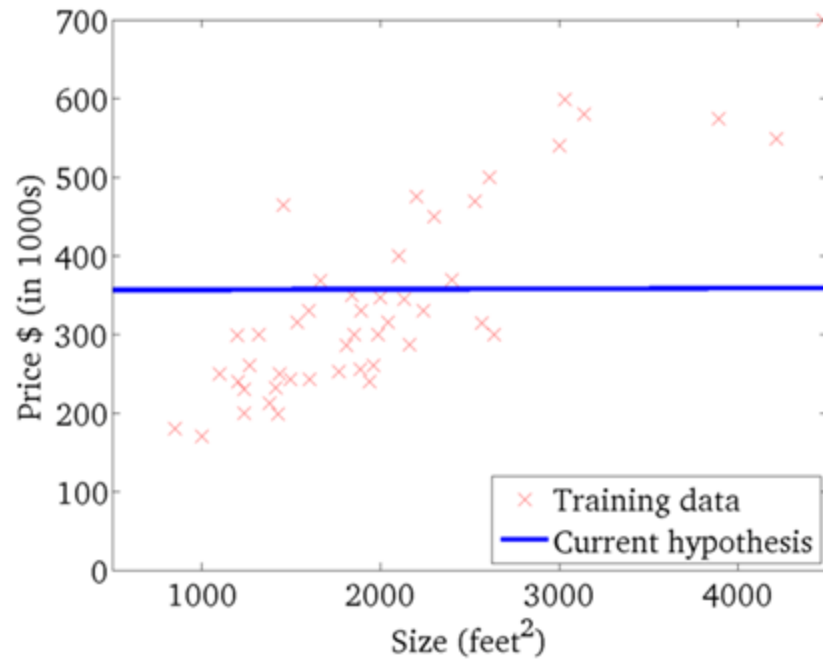
(function of the parameters θ_0, θ_1)



Gradient Descent

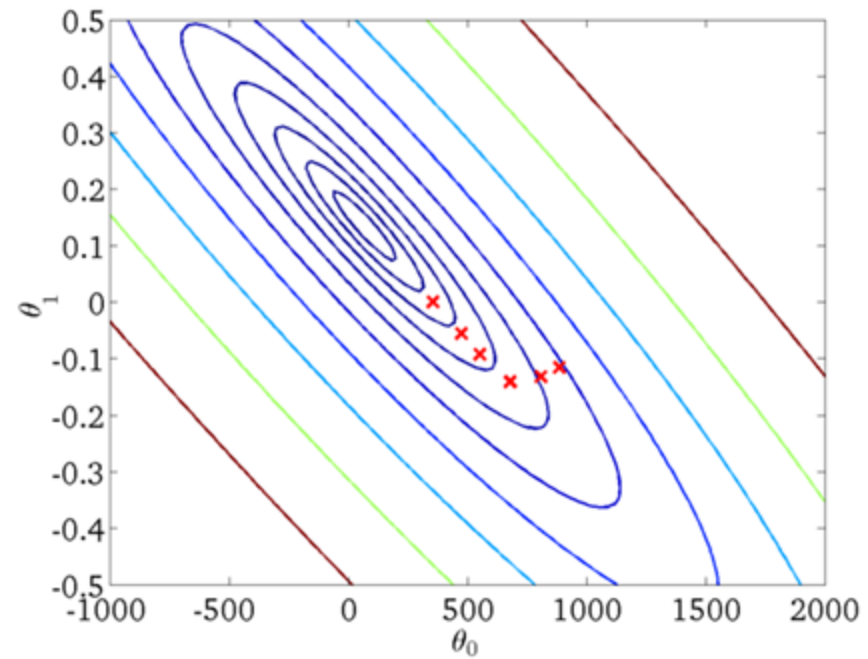
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

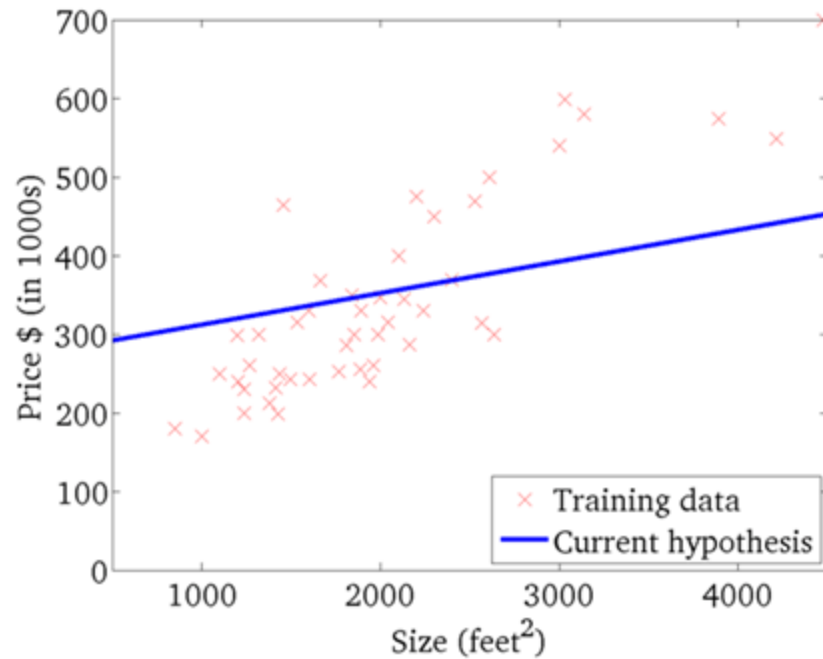
(function of the parameters θ_0, θ_1)



Gradient Descent

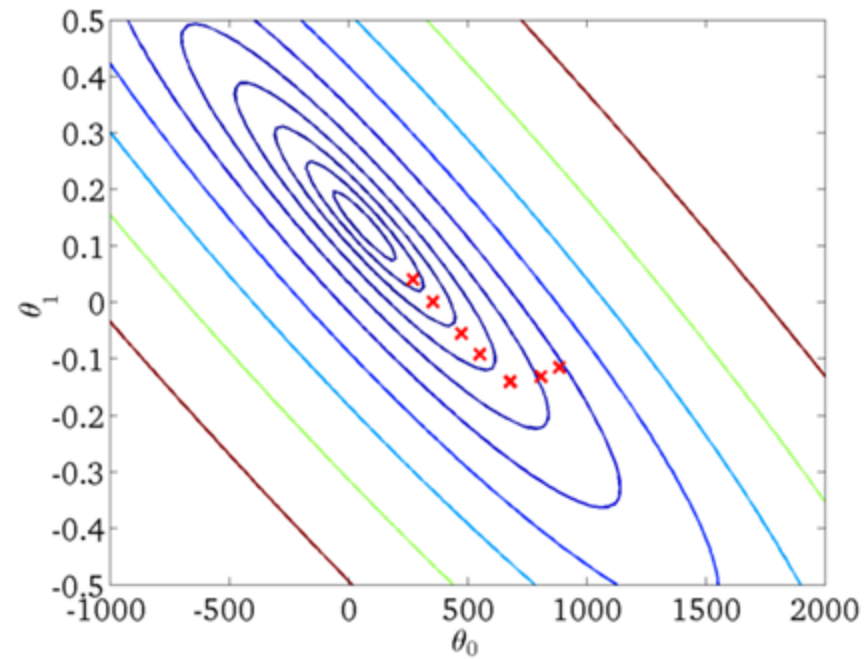
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

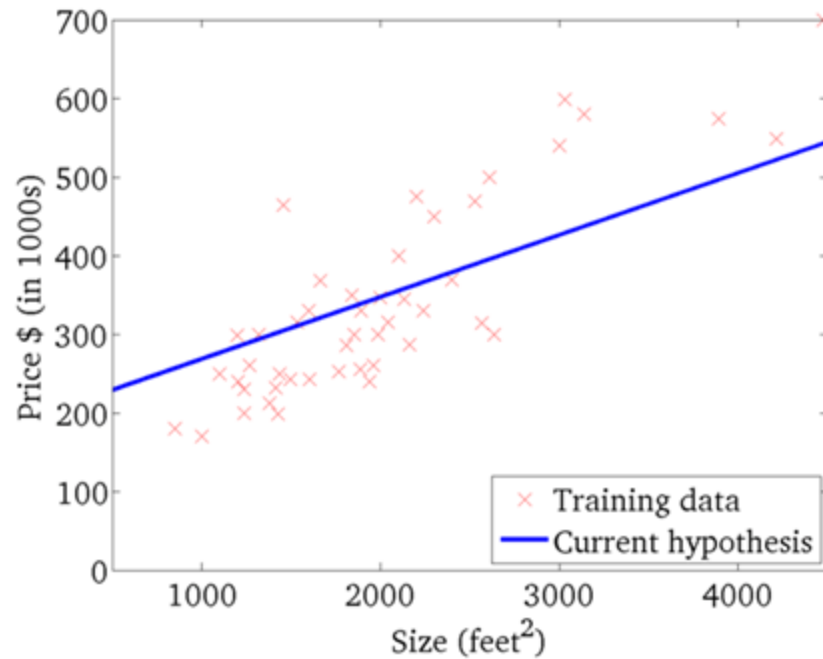
(function of the parameters θ_0, θ_1)



Gradient Descent

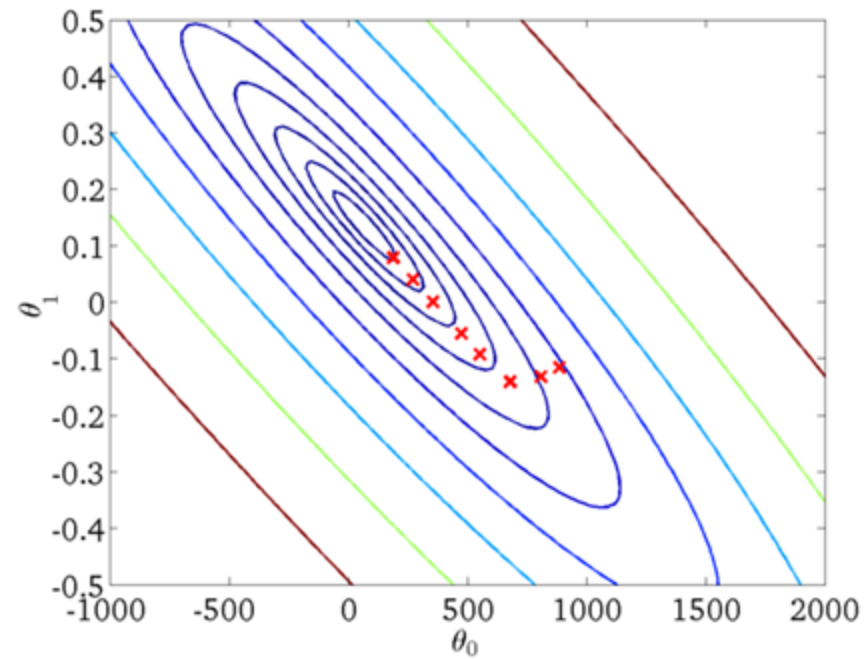
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

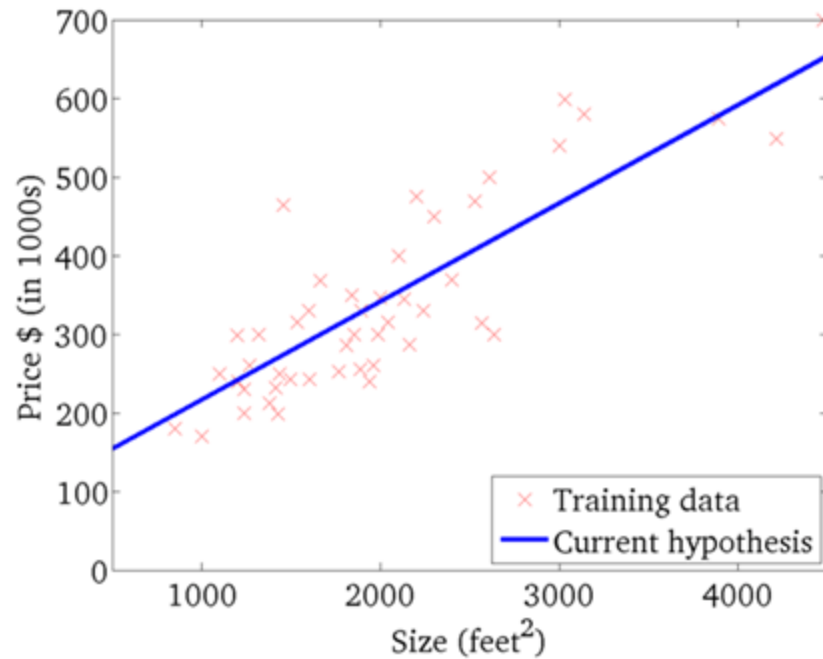
(function of the parameters θ_0, θ_1)



Gradient Descent

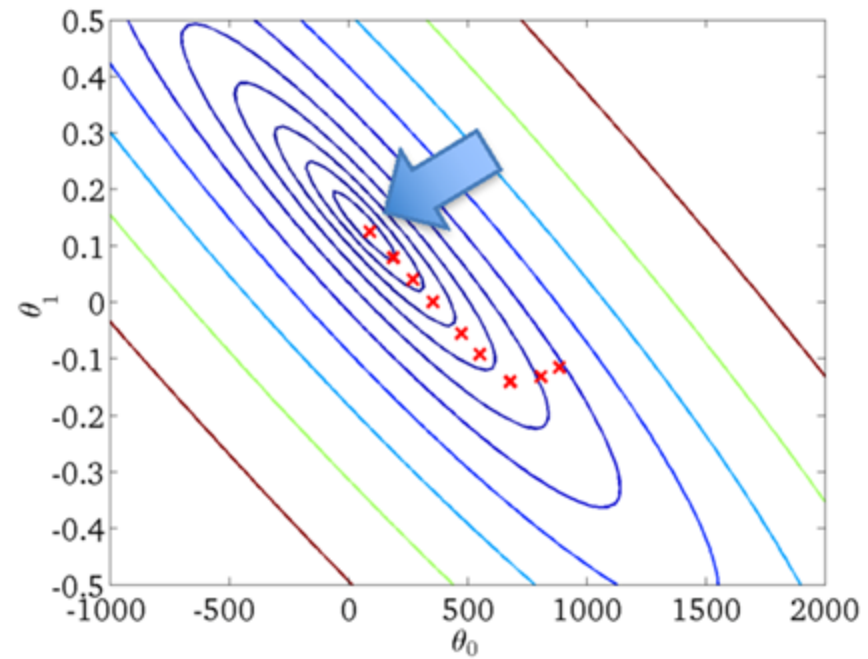
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



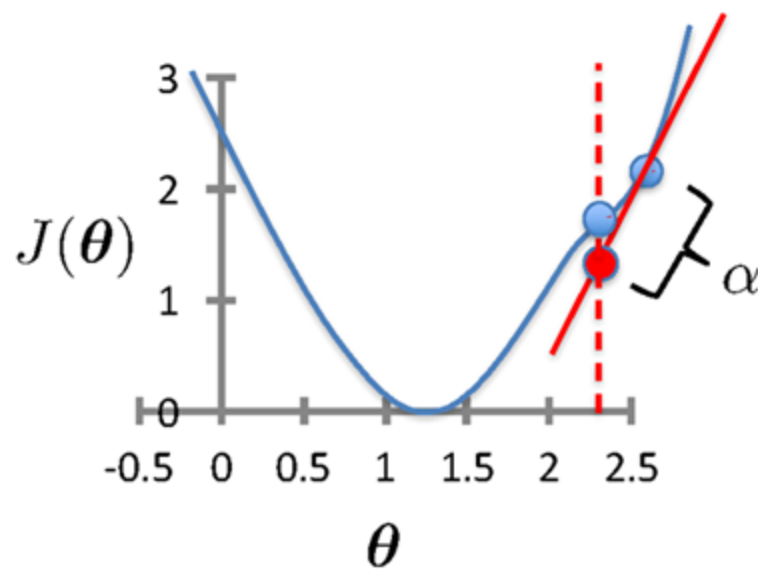
Gradient Descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

learning rate (small)
e.g., $\alpha = 0.05$



Gradient Descent

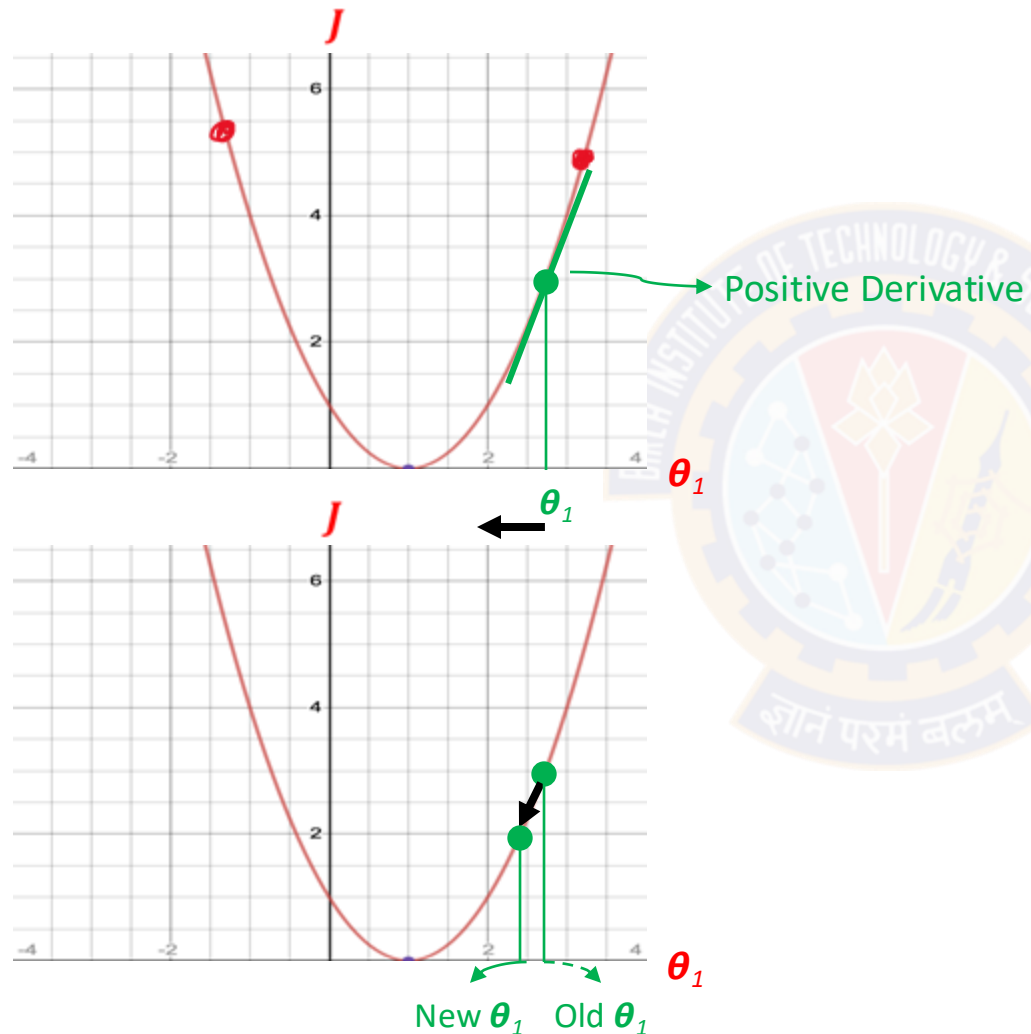
- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

For Linear Regression:
$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 \\ &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^n \left(\sum_{k=0}^d \theta_k x_k^{(i)} - y^{(i)} \right)^2 \\ &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{k=0}^d \theta_k x_k^{(i)} - y^{(i)} \right) \times \frac{\partial}{\partial \theta_j} \left(\sum_{k=0}^d \theta_k x_k^{(i)} - y^{(i)} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{k=0}^d \theta_k x_k^{(i)} - y^{(i)} \right) x_j^{(i)} \\ &\dots \end{aligned}$$

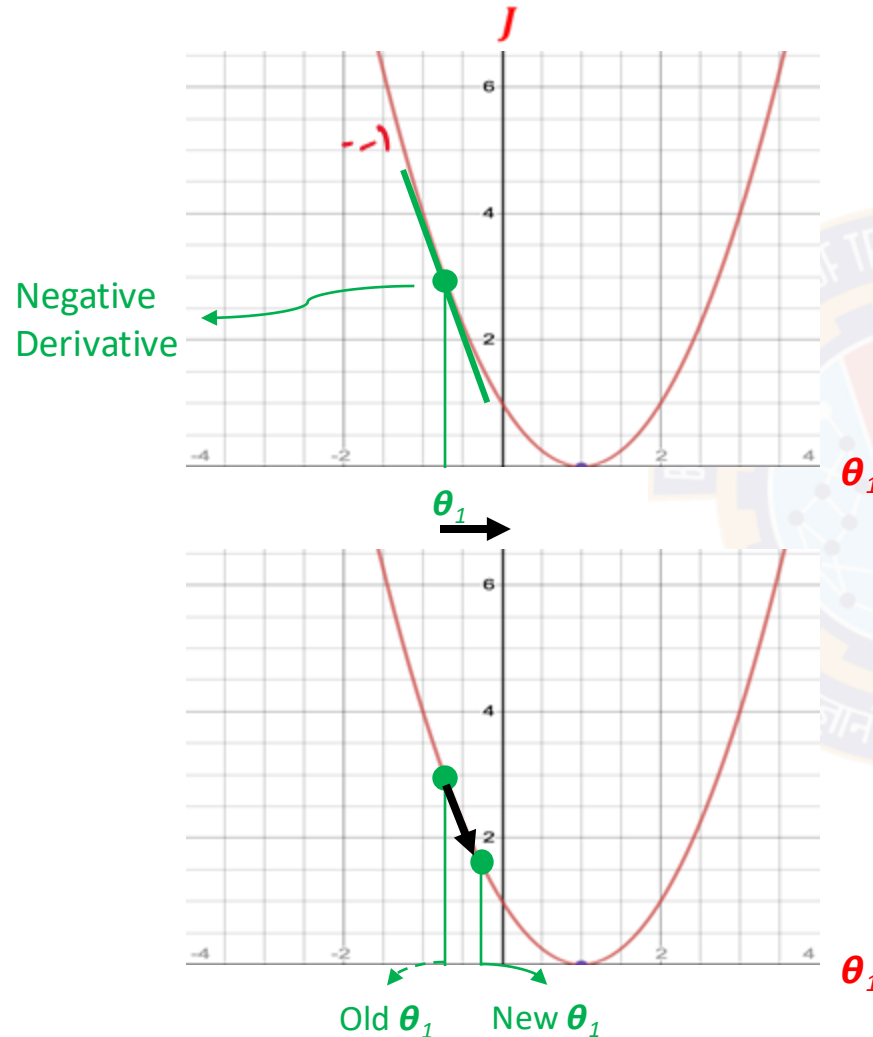
Guarantee of Convergence



$$\begin{aligned}\theta_1 &= \theta_1 - \alpha \frac{dJ(\theta_1)}{d\theta_1} \\ &= \theta_1 - \alpha (\text{Positive Number}) \\ &\text{Decrease } \theta_1 \text{ by a certain value}\end{aligned}$$

Source Credit: Prof. Mohammad Hammoud

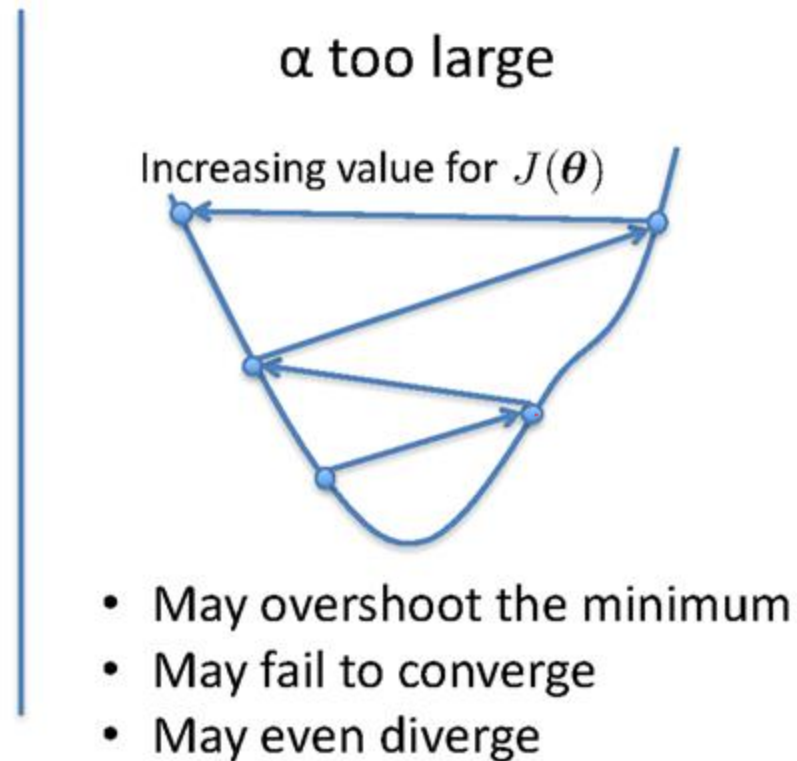
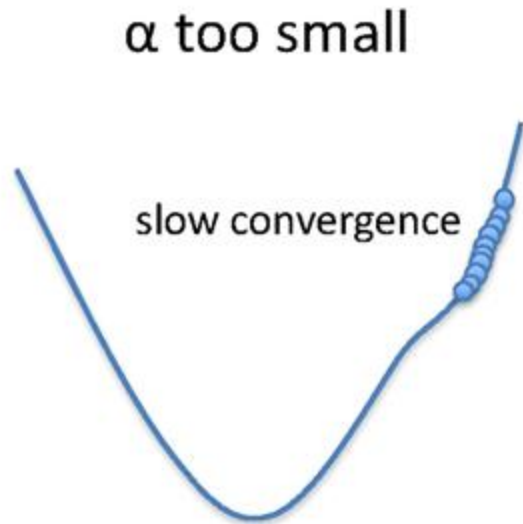
Guarantee of Convergence



$$\begin{aligned}\theta_1 &= \theta_1 - \alpha \frac{dJ(\theta_1)}{d\theta_j} \\ &= \theta_1 - \alpha (\text{Negative Number}) \\ &\text{Increase } \theta_1 \text{ by a certain value}\end{aligned}$$

Source Credit: Prof. Mohammad Hammoud

Choosing Learning Rate



To see if gradient descent is working, print out $J(\theta)$ each iteration

- The value should decrease at each iteration
- If it doesn't, adjust α

Gradient Descent algorithm : Effect of Feature Scaling

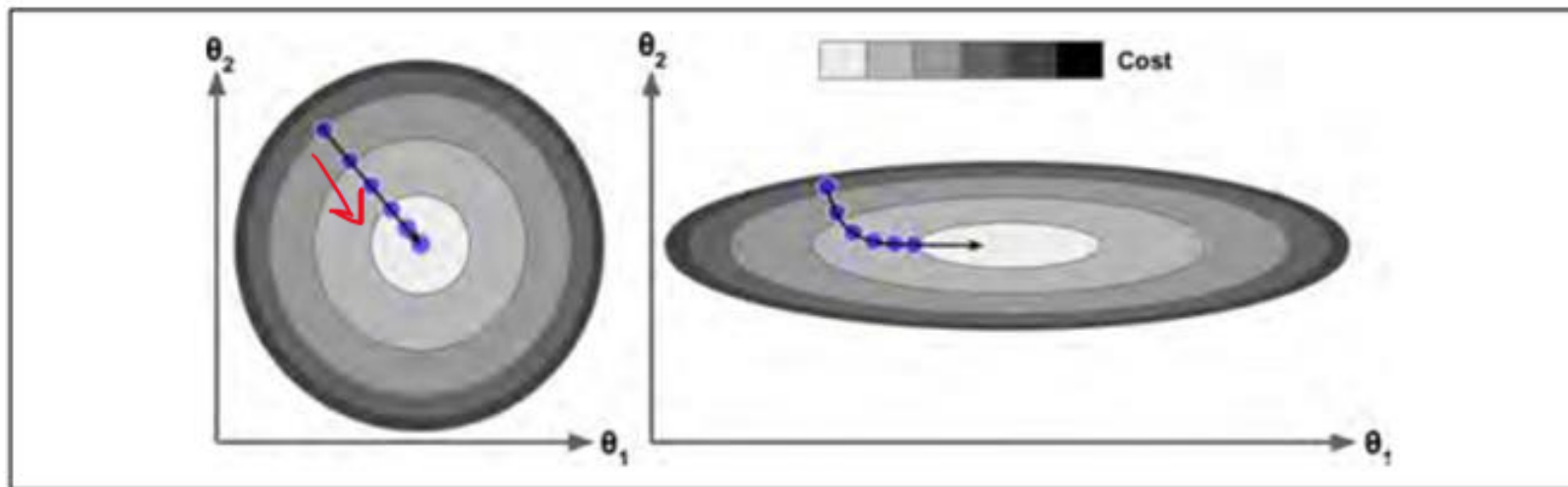


Figure 4-7. Gradient Descent with and without feature scaling

Aspect	With Scaling	Without Scaling
Contours shape	Circular	Elongated
Convergence speed	Fast	Slow
Gradient descent path	Straight to minimum	Zigzagging, inefficient

Why this happens?

Gradient descent uses the **slope** (partial derivatives) to update each parameter θ_j . When features have very different scales:

- One direction (e.g., θ_1) might change much faster than another (e.g., θ_2).
- This causes **inefficient updates**, as the algorithm overcorrects in one direction and undercorrects in another.
- The result is a **zigzagging path** instead of a direct descent.

Solution: Feature Scaling

Feature Scaling ensures:

- All features are on a **similar scale**, typically by:
 - **Standardization** (zero mean, unit variance)
 - **Min-max normalization** (rescale to $[0, 1]$)
- This transforms the cost function contours into **circular shapes**.
- Gradient descent moves more **directly** toward the optimum.

Gradient Descent algorithm : Effect of Learning Rate

Problem Type 2 – Interpretation of Convergence or Effect of Hyper parameters

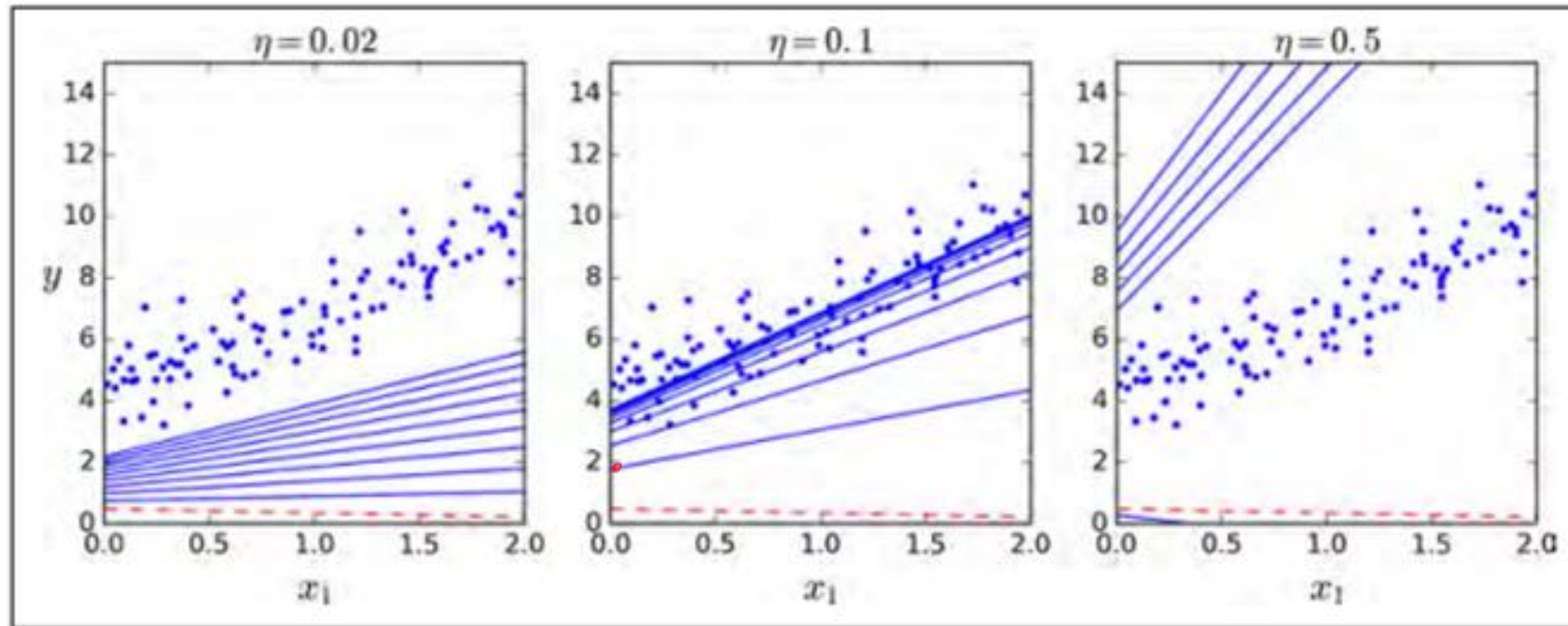


Figure 4-8. Gradient Descent with various learning rates

Gradient Descent for Linear Regression

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} \quad \begin{array}{l} \text{simultaneous} \\ \text{update} \\ \text{for } j = 0 \dots d \end{array}$$

- To achieve simultaneous update
 - At the start of each GD iteration, compute $h_{\theta} \left(\mathbf{x}^{(i)} \right)$
 - Use this stored value in the update step loop
- Assume convergence when $\|\theta_{new} - \theta_{old}\|_2 < \epsilon$

$$\text{L}_2 \text{ norm: } \|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_{|\mathbf{v}|}^2}$$

$$\mathbf{v} = (v_1, v_2, v_3 \dots v_n)$$

Fit a linear Regression Line :

Gradient Descent : Problem Type 3

- Steps :

- (Assuming : 'n' no.of. instances and two predictors {x1, x2} and linear regression)

1. Identification of the equations $y = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

2. Cost function & derivative

$$\theta'_0 = \theta_0 - \frac{1}{n} * \text{learning rate} * (\text{sum } (\theta_0 + \theta_1 x_1 + \theta_2 x_2 - y))$$

$$\theta'_1 = \theta_1 - \frac{1}{n} * \text{learning rate} * (\text{sum } (\theta_0 + \theta_1 x_1 + \theta_2 x_2 - y) * x_1)$$

$$\theta'_2 = \theta_2 - \frac{1}{n} * \text{learning rate} * (\text{sum } (\theta_0 + \theta_1 x_1 + \theta_2 x_2 - y) * x_2)$$

3. Apply the equations

Numerical Example

Tiny Dataset: Predict house price from size

Size (x_1)	Price (y)
1	2
2	4
3	5

Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \in \mathbb{R}^3, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \in \mathbb{R}^2$$

Numerical Example

Hyperparameters: $\eta = 0.1$ $N = 3$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Vectorized Prediction: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Initial predictions: $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

Loss Function: $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

Initial loss: $J(\mathbf{w}^{(0)}) = \frac{1}{6} \left\| \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\|^2 = \frac{1}{6}(4 + 16 + 25) = 7.5$

Numerical Example: Initialization

Hyperparameters: $\eta = 0.1$ $N = 3$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Vectorized Prediction: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Initial predictions: $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

Loss Function: $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

Initial loss: $J(\mathbf{w}^{(0)}) = \frac{1}{6} \left\| \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\|^2 = \frac{1}{6}(4 + 16 + 25) = 7.5$

Numerical Example: Computing Gradient

Error vector: $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$

Gradient computation: $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N} \mathbf{X}^T \mathbf{e}^{(0)}$

$$\begin{aligned} \nabla J(\mathbf{w}^{(0)}) &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix} \\ &= \frac{1}{3} \begin{bmatrix} -2 - 4 - 5 \\ -2 - 8 - 15 \end{bmatrix} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} \end{aligned}$$

Numerical Example: Weight Update

Gradient descent update: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J(\mathbf{w}^{(0)})$

Computing new weights: $\mathbf{w}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}$

New predictions: $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)}$

$$\hat{\mathbf{y}}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}$$

Numerical Example: New Loss

New error vector: $\mathbf{e}^{(1)} = \hat{\mathbf{y}}^{(1)} - \mathbf{y} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.80 \\ -1.97 \\ -2.13 \end{bmatrix}$

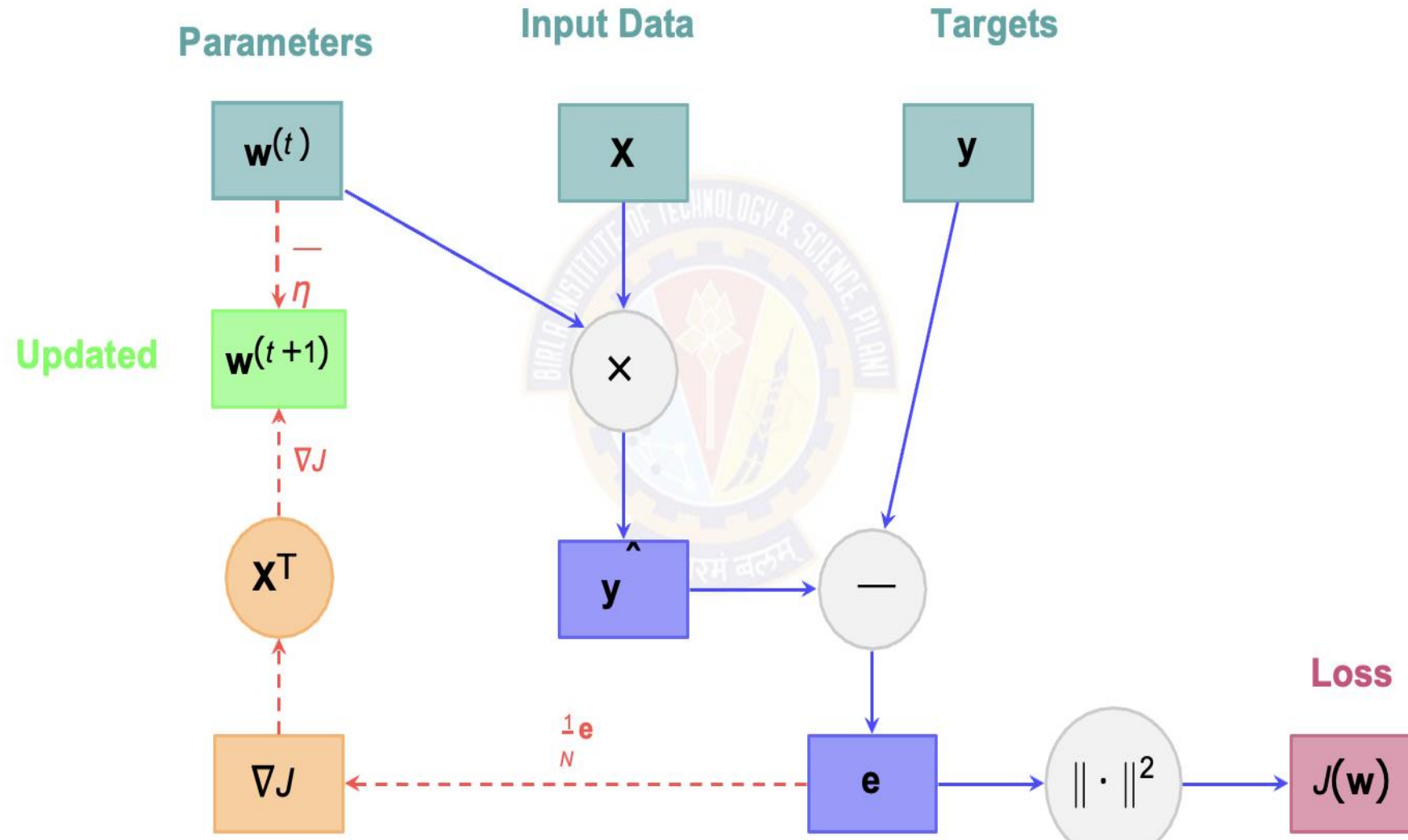
Loss after one iteration: $J(\mathbf{w}^{(1)}) = \frac{1}{6} \|\mathbf{e}^{(1)}\|^2 = \frac{1}{6}(0.64 + 3.88 + 4.54) \approx 1.51$

Progress:

- Initial loss: $J(\mathbf{w}^{(0)}) = 7.5$
- After 1 iteration: $J(\mathbf{w}^{(1)}) = 1.51$
- **80% reduction!** ✓

Continue until $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations reached

Computational Graph for One gradient



Fit a linear Regression Line :Gradient Descent

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of 0.02 for the following data , if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as 5 and the slope of independent variables as -0.03 (negative).

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

1. Identification of the equations: $RRCHD = 5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure$
2. Cost function & derivative
 1. $\theta'_0 = \theta_0 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD))$
 $= 5 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD))$
 2. $\theta'_1 = \theta_1 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD) * BMI)$
 $= -0.03 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD) * BMI)$
 3. $\theta'_2 = \theta_2 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD) * Diastolic\ Pressure)$
 $= -0.03 - 1/3 * 0.02 * (\text{sum}(5 - 0.03 * BMI - 0.03 * Diastolic\ Pressure - RRCHD) * Diastolic\ Pressure)$
3. Apply the equations : Answer at the end of first iteration:
 $\theta_0 = 5.0016$, $\theta_1 = 0.0476$, $\theta_2 = 0.179$

Closed Form Solution Vs. Gradient Descent

Gradient Descent	Closed Form Solution
• Requires multiple iterations • Need to choose α • Works well when n is large • Can support incremental learning	• Non-iterative • No need for α • Slow if n is large – Computing $(X^T X)^{-1}$ is roughly $O(n^3)$



Types of Gradient Descent Algorithms



Gradient Descent: Variants

- **Batch** gradient descent refers to calculating the derivative from all training data before calculating an update.
- **Minibatch** refers to calculating derivative of mini groups of training data before calculating an update.
- **Stochastic** gradient descent refers to calculating the derivative from each training data instance and calculating the update immediately

Diagram illustrating the Batch Gradient Descent algorithm. The algorithm repeats until convergence, calculating the derivative of the cost function $J(\theta_0, \theta_1)$ with respect to θ_0 and θ_1 simultaneously.

repeat until convergence {

$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$

$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$

}

Update θ_0 and θ_1 simultaneously

$temp0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$

$temp1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$

$\theta_0 := temp0$

$\theta_1 := temp1$

The diagram includes a watermark of the Birla Institute of Technology & Science (PILANI) logo in the background. Annotations include a pink box around the θ_0 update formula, a blue box around the θ_1 update formula, and a yellow box for the simultaneous update instruction. Arrows point from the derivative terms $\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$ and $\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$ to their respective formulas.

Gradient Descent: Variants

- **Batch** gradient descent refers to calculating the derivative from all training data before calculating an update.

Initialize the Parameters $(\theta_0^1, \theta_1^1, \dots)$

$K=1$

Repeat until Convergence {

$$\theta_0^{k+1} = \theta_0^k - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1^{k+1} = \theta_1^k - \frac{\alpha}{m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)}) * x_1^{(i)})$$

$$\theta_2^{k+1} = \theta_2^k - \frac{\alpha}{m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)}) * x_2^{(i)})$$

.....

$K=k+1$

}

return $(\theta_0^1, \theta_1^1, \dots)$

Gradient Descent: Variants

- Minibatch refers to calculating derivative of mini groups of training data before calculating an update.

Divide the training instances into “N” batches each of size “m”
Initialize the Parameters $(\theta_0^1, \theta_1^1, \dots)$
 $K=1$
Repeat until Convergence {
 Repeat for every batch in $1 : N$, each with ‘m’ instances {
 $\theta_0^{k+1} = \theta_0^k - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$
 $\theta_1^{k+1} = \theta_1^k - \frac{\alpha}{m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)}) * x_1^{(i)})$
 $\theta_2^{k+1} = \theta_2^k - \frac{\alpha}{m} \sum_{i=1}^m ((h_{\theta}(x^{(i)}) - y^{(i)}) * x_2^{(i)})$

 $K=k+1$
 }
}
return $(\theta_0^1, \theta_1^1, \dots)$

Gradient Descent: Variants

- **Stochastic gradient** descent refers to calculating the derivative from each training data instance and calculating the update immediately

Randomly shuffle training instances

Initialize the Parameters $(\theta_0^1, \theta_1^1, \dots)$

$K=1$

Repeat until Convergence {

Sample with replacement, only one random training instance “i” at a time

$$\theta_0^{k+1} = \theta_0^k - \alpha (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1^{k+1} = \theta_1^k - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) * x_1^{(i)}$$

$$\theta_2^{k+1} = \theta_2^k - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) * x_2^{(i)}$$

.....

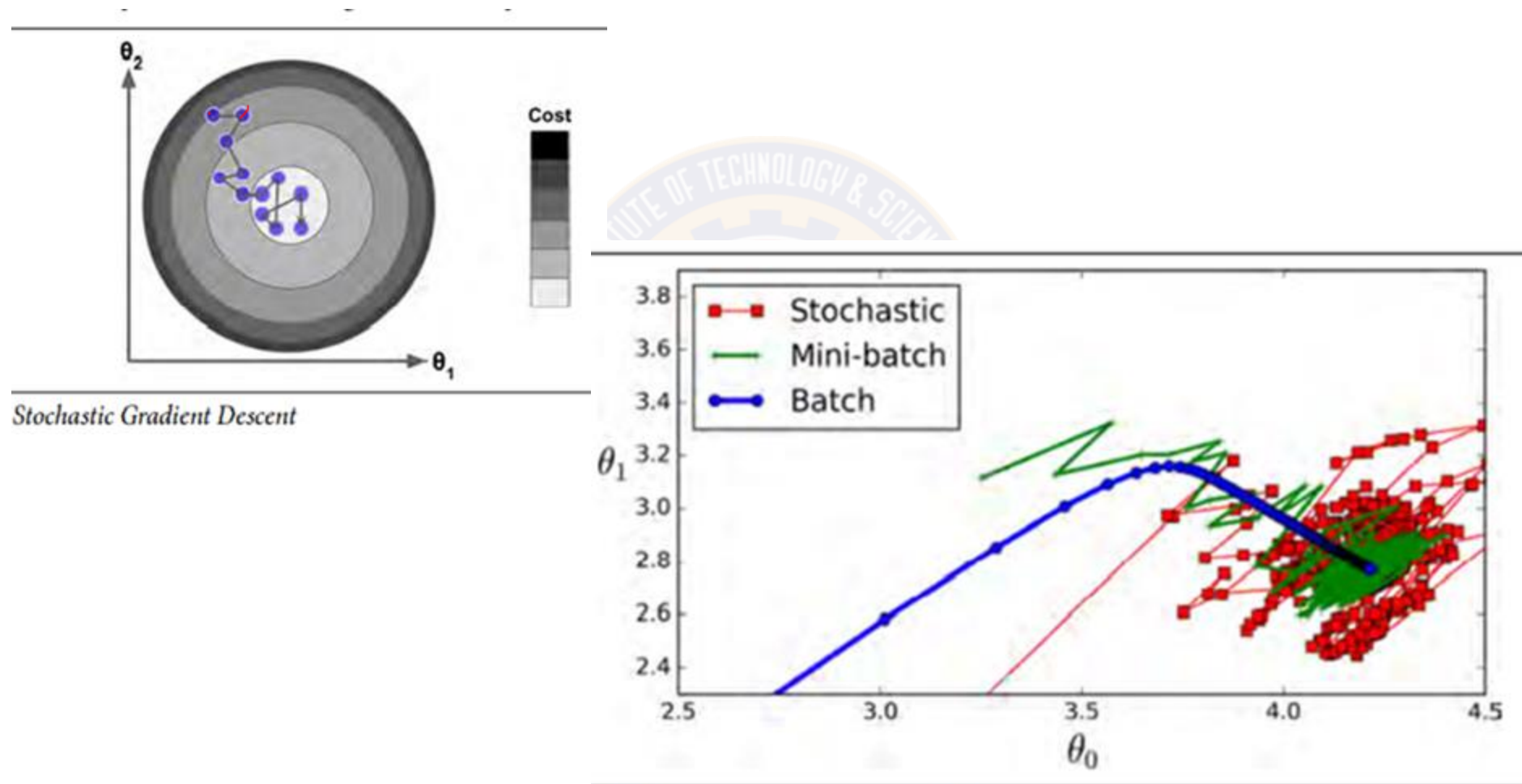
$K=k+1$

}

}

return $(\theta_0^1, \theta_1^1, \dots)$

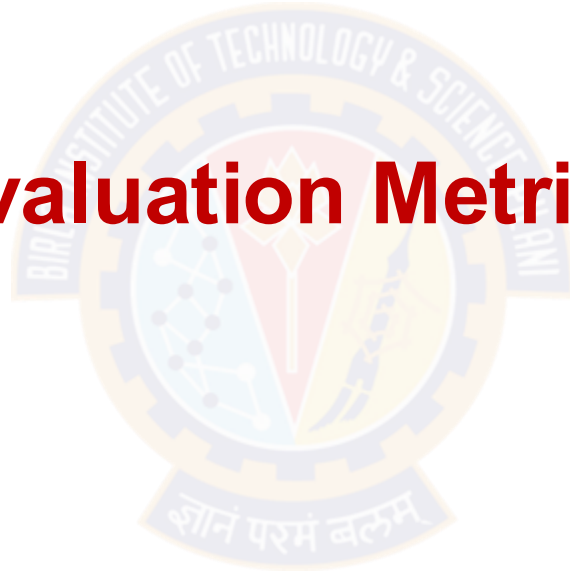
Gradient Descent: Variants - Effect on Convergence



1. Gradient Descent paths in parameter space

Type	Data Used per Update	Updates per Epoch	Speed	Stability	Common Use Case
Batch	All data	1	Slow	High	Small datasets
Stochastic	1 sample	n (number of samples)	Fast	Low	Online learning
Mini-Batch	Small batches	n / batch_size	Medium	Medium	Deep learning (default)

Evaluation Metrics



Evaluation of Linear Regression Model

Mileage (in kmpl)	Car Price (in cr)
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51
....	

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}|$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2}$$

$$R^2 = 1 - \frac{SS_{residual}}{SS_{Total}}$$

Unseen Data	
Mileage (in kmpl)	Car Price (in cr)
7.5	9.25
10	6.5
....	

Model 1

$$\text{CarPrice} = 8.5 + 0.5 \text{ Mileage} - 1.5 \text{ Mileage}^2$$

Model 2

$$\text{CarPrice} = -5.5 + 1.5 \text{ Mileage}$$

Evaluation of Linear Regression Model

R-squared

Mileage (in kmpl)	Car Price (in cr)
9.8	10.48
9.12	1.75
9.5	6.95
10	2.51
....	

variation in 'y' that is explained by a regression model

$$\text{explained variation} = \hat{y} - \bar{y}$$

variation in 'y' that is not captured/explained by a regression model

$$\text{unexplained variation} = y - \hat{y}$$

$$\text{total variation} = (y - \hat{y}) + (\hat{y} - \bar{y}) = (y - \bar{y})$$

$$R^2 = 1 - \frac{SS_{\text{residual}}}{SS_{\text{Total}}}$$

Model 1

$$\text{CarPrice} = 8.5 + 0.5 \text{ Mileage} - 1.5 \text{ Mileage}^2$$

- variation that is explained by a regression model**
- measures the goodness of fit of a regression model**

Car Price (in cr)
10.48
1.75
6.95
2.51
Mean Y

$$SS_{\text{explained}} = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

$$SS_{\text{residual}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$SS_{\text{Total}} = \sum_{i=1}^n (y_i - \bar{y})^2$$

where :

$SS_{\text{explained}}$ = explained variation sum of squares

SS_{residual} = unexplained variation sum of squares

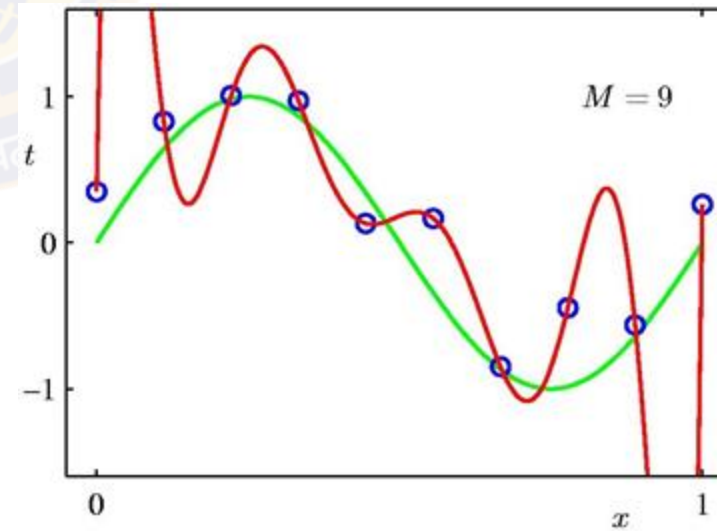
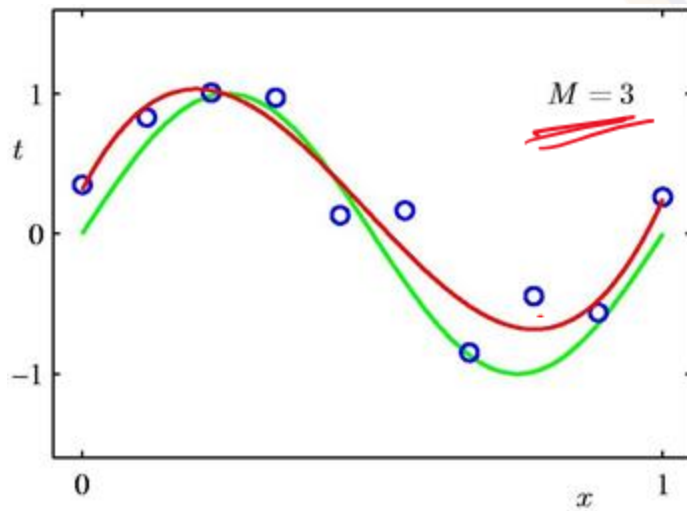
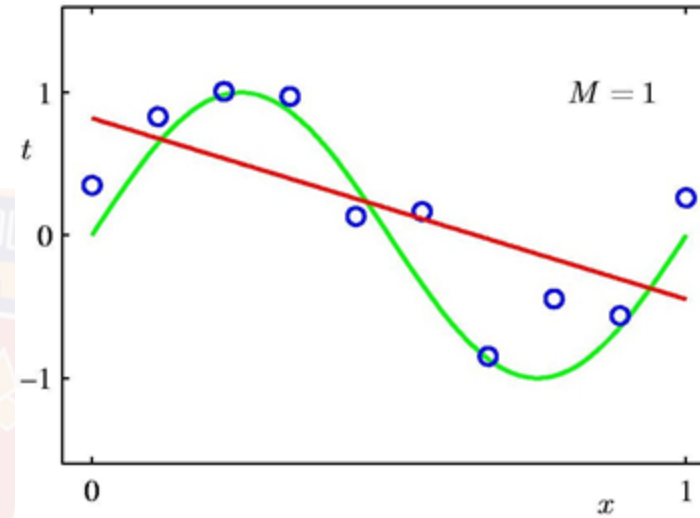
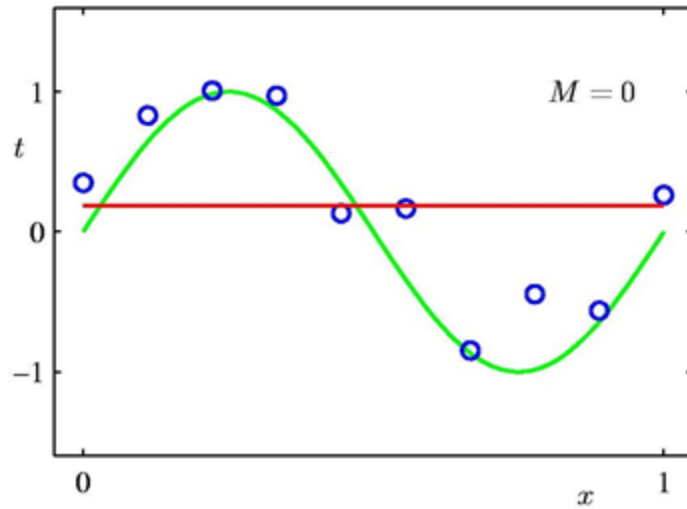
SS_{Total} = total variation sum of squares



Linear Basis Models

What if output is a non-linear function of input vector?

Polynomial Regression



Linear Basis Function Models

- The inputs **X** for linear regression can be:
 - Original quantitative inputs
 - Transformation of quantitative inputs
 - e.g. log, exp, square root, square, etc.
 - Polynomial transformation
 - example: $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \beta_3 \cdot x^3$
 - Basis expansions
 - Dummy coding of categorical inputs
 - Interactions between variables
 - example: $x_3 = x_1 \cdot x_2$

This allows use of linear regression techniques to fit non-linear datasets.

Input X	Output Y
exp(2)	
exp(4)	
exp(6.3)	
exp(9.2)	

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

X1 = Graduate	X2 = PostGraduate	X3 = Others	Y Salary Of the Employee
0	0	1	2
1	0	0	3
0	0	1	4
0	1	0	5
1	0	0	6

Linear Basis Function Models

Example: an M-th order polynomial function of one dimensional feature x :

$$y(x, \mathbf{w}) = w_0 + \sum_{j=1}^M w_j x^j$$

where x^j = j-th power of x

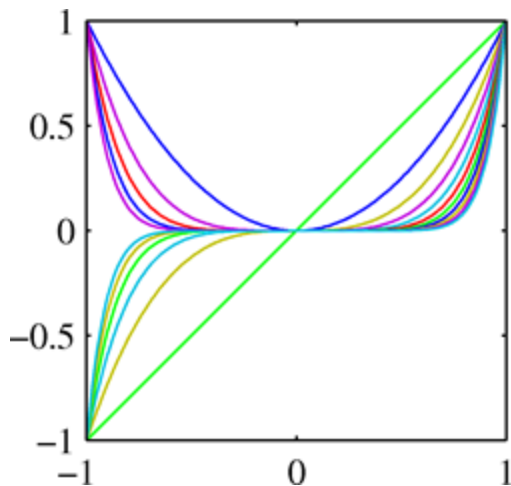
$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M = \sum_{j=0}^M w_j x^j$$

X No.of.Years of Experience (in Years)	X^2	Y Salary Of the Employee (in Lakhs)
1	1	2
2	4	3
3	9	4
4	16	5
5	25	6

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$$

- $\Phi_j(x)$ are known as *basis functions*. Typically, $\Phi_0(x) = 1$, so that w_0 acts as a bias.
- In the simplest case, we use linear basis functions : $\Phi_d(x) = x^d$.
- They are called linear models because this function is linear in $\mathbf{w}$.

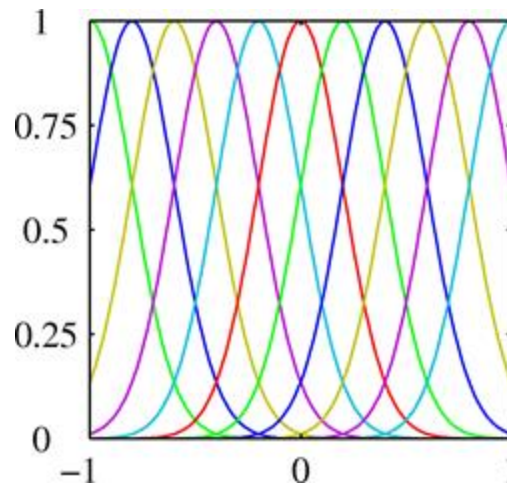
Linear Basis Function Models - Examples



Polynomial basis functions:

$$\phi_j(x) = x^j.$$

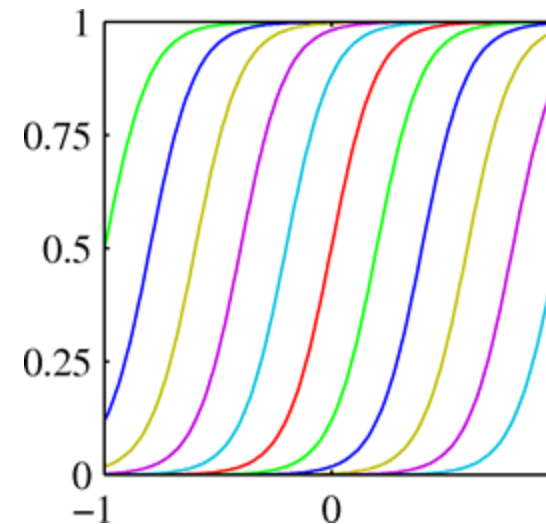
These are global; a small change in x affect all basis functions.



Gaussian basis functions:

$$\phi_j(x) = \exp \left\{ -\frac{(x - \mu_j)^2}{2s^2} \right\}$$

These are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (width).



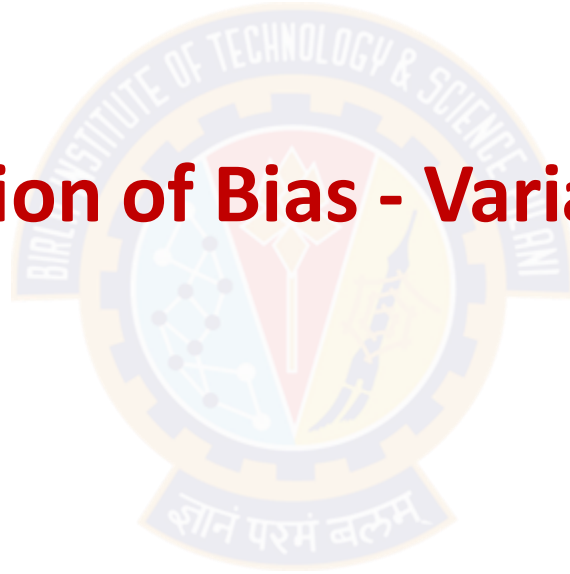
Sigmoidal basis functions:

where $\phi_j(x) = \sigma \left(\frac{x - \mu_j}{s} \right)$

$$\sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Also these are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (slope).

Notion of Bias - Variance



Bias and Variance

1. What is Bias?

- **Bias** = Error from wrong assumptions in the model.
- It tells how far the model's predictions are from the *true relationship*.

High Bias means the model is **too simple** (it underfits).

2. What is Variance?

- **Variance** = Error from sensitivity to training data.
- It tells how much the model's predictions would change if we trained on a different dataset.

High Variance means the model is **too complex** (it overfits).

- Example: A very wiggly polynomial that perfectly fits training points but fails on new data.

Underfitting and Overfitting

Underfitting

- Happens when the model is **too simple** to capture the patterns in the data.
- It has **high bias** (wrong assumptions) and usually **low variance** (stable but inaccurate).

Symptoms:

- Both **training error** and **test error** are high.
- Model cannot learn even training data well.

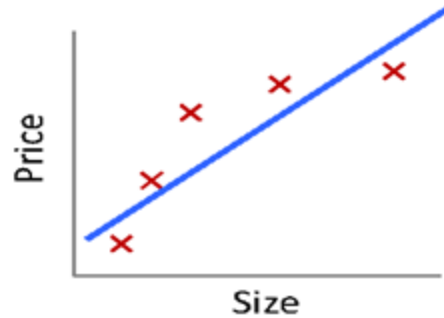
Overfitting

- Happens when the model is **too complex** and tries to learn not only the true pattern but also the **noise** in the training data.
- It has **low bias** (fits training data very well) but **high variance** (fails on new/unseen data).

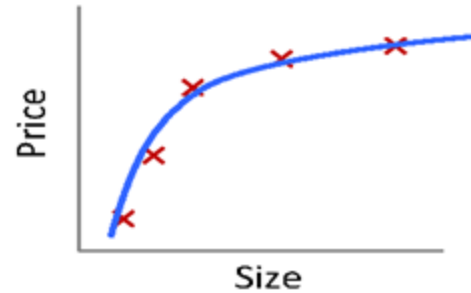
Symptoms:

- **Training error is very low.**
- **Test error is very high** (poor generalization).

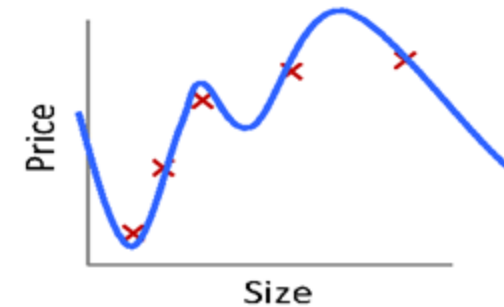
Quality of Fit



$\theta_0 + \theta_1 x$
Underfitting
(high bias)



$\theta_0 + \theta_1 x + \theta_2 x^2$
Correct fit



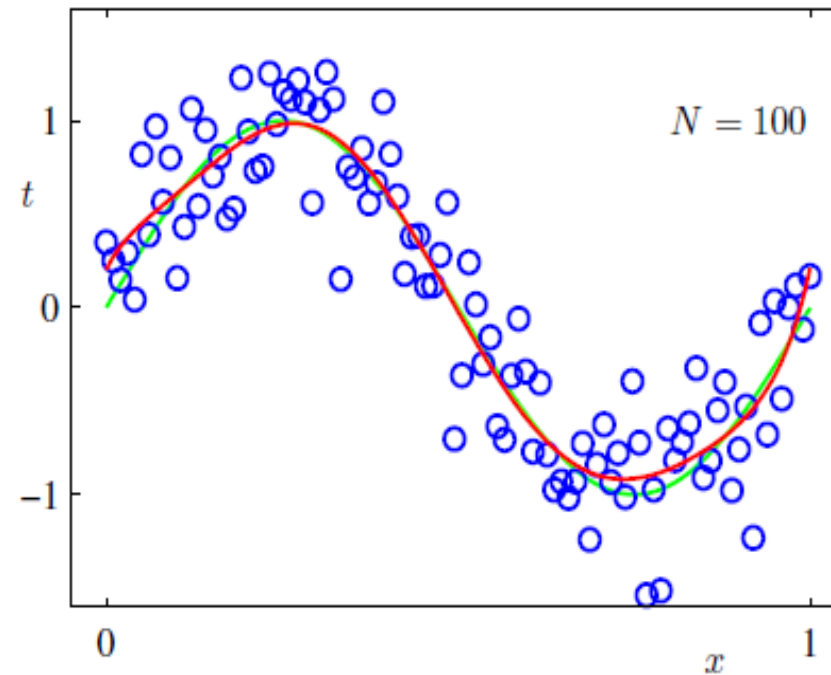
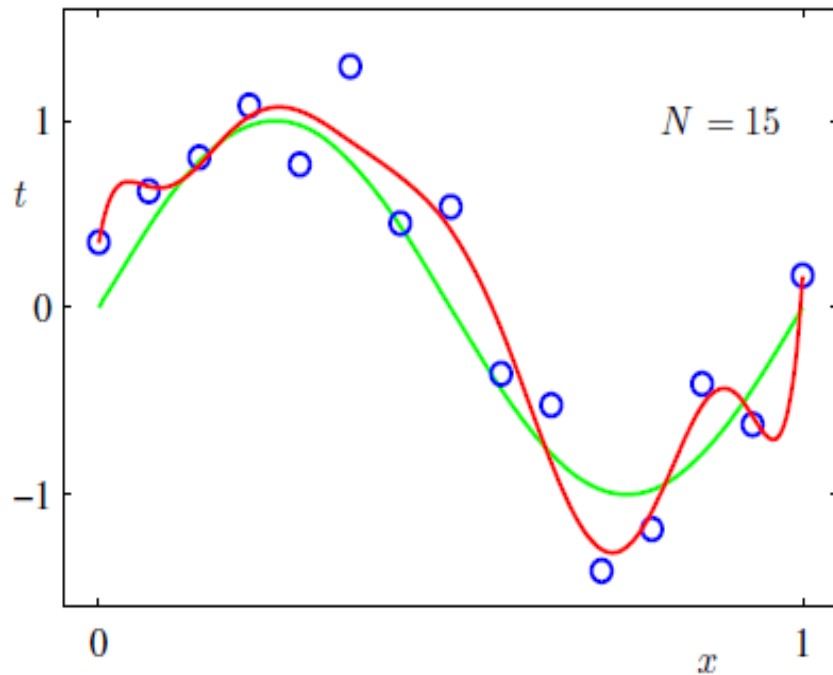
$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
Overfitting
(high variance)

Overfitting:

- The learned hypothesis may fit the training set very well ($J(\theta) \approx 0$)
- ...but fails to generalize to new examples

Handling Overfitting – Way 1

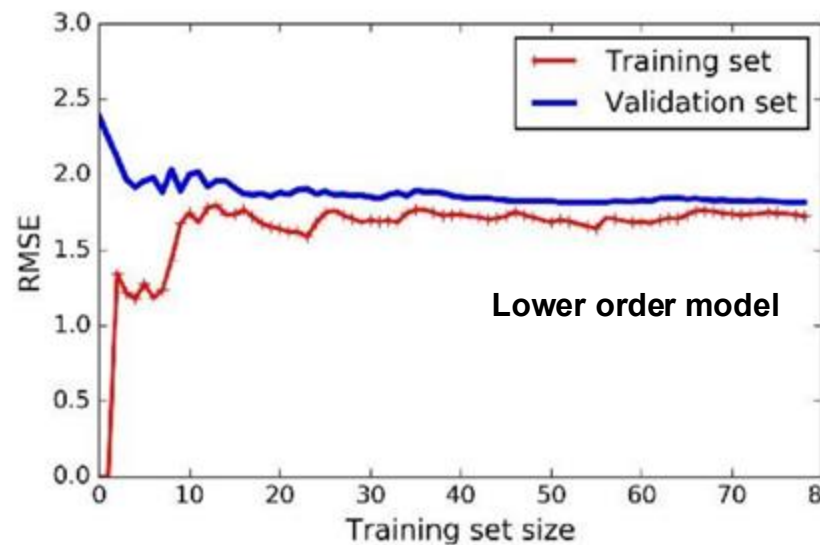
- Increase in Size of the data set reduces the over-fitting problem



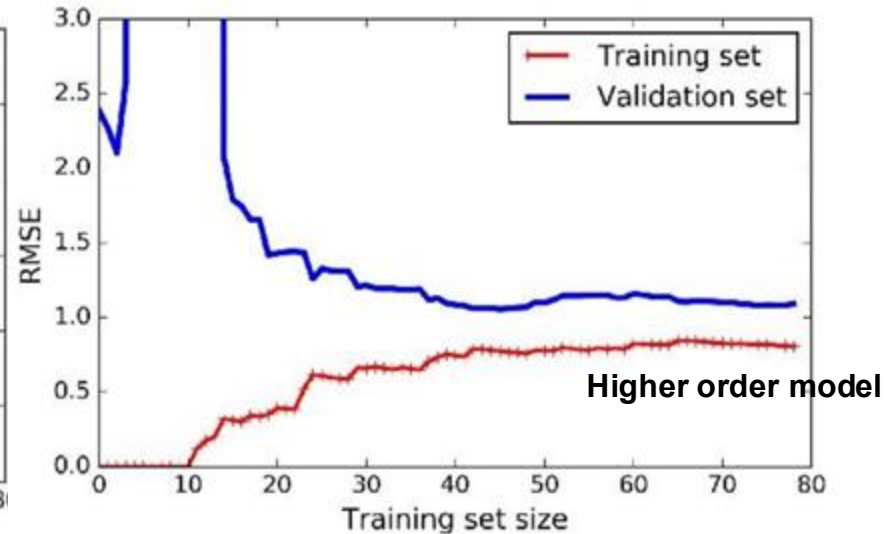
Effect of Training Size on Over fitting

Problem Type 4 : Interpretation of the Model Fit

- Size of training dataset needs to be large to prevent when higher order model is used.



This model is suffering from high bias.
It is underfitting the data



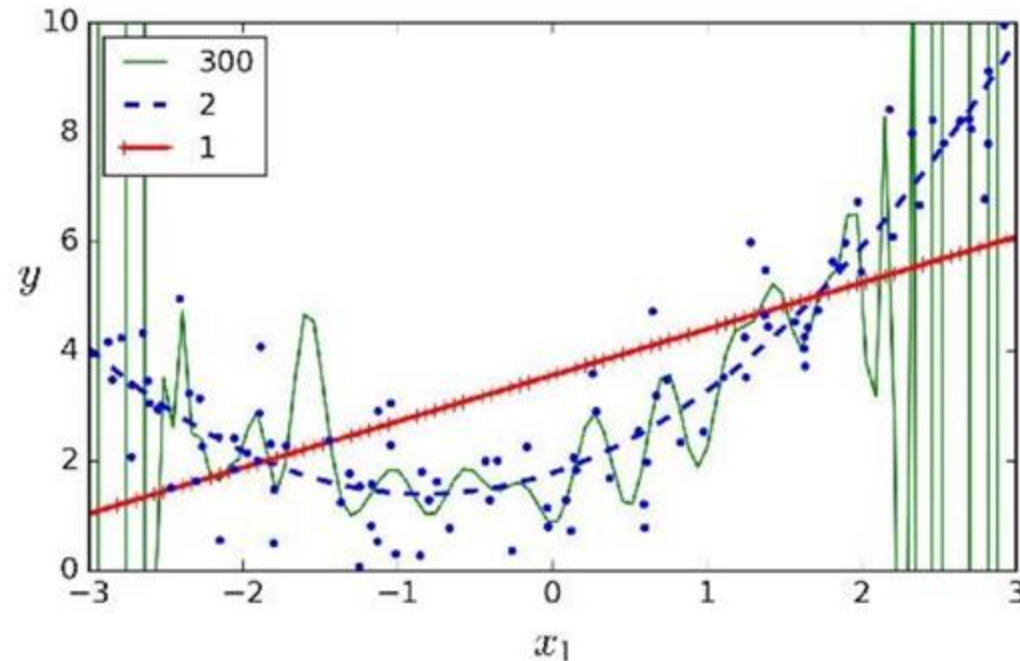
- This model has **high variance**
- It is **overfitting**

Polynomial Fitting can lead to Overfitting

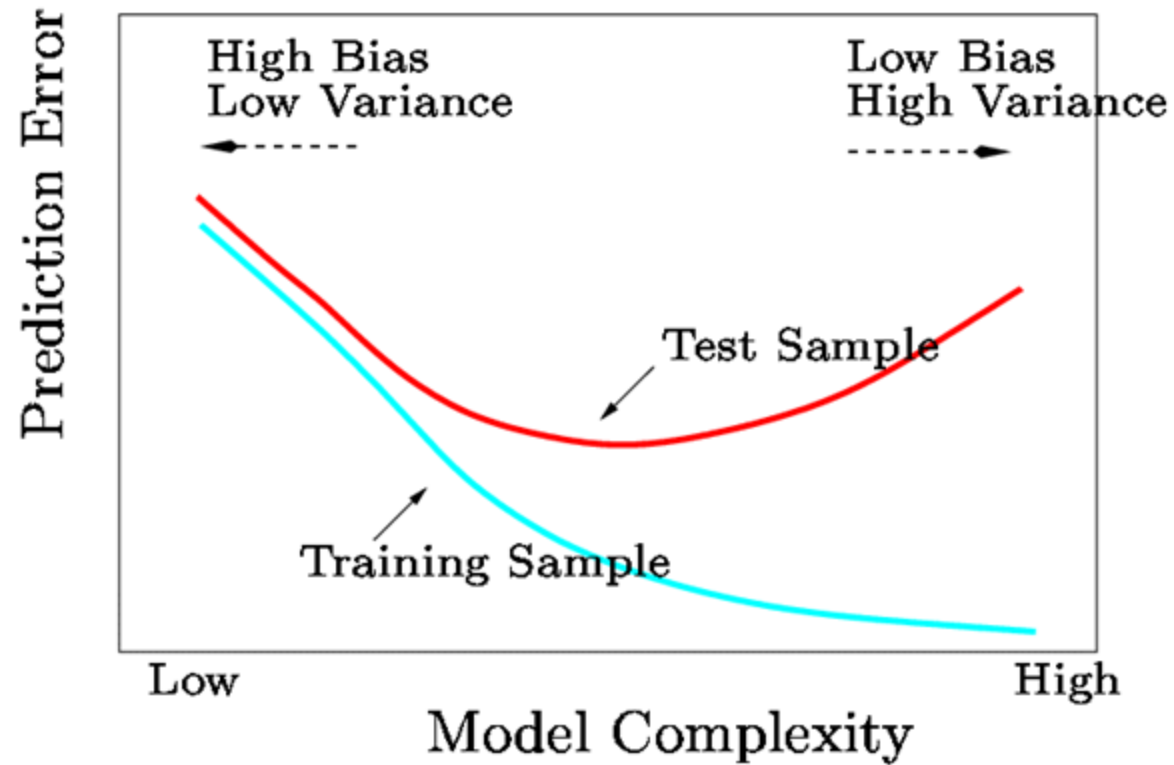
Handling Overfitting – Way 2 – Reduce the complexity of the model

Underlying target function is quadratic

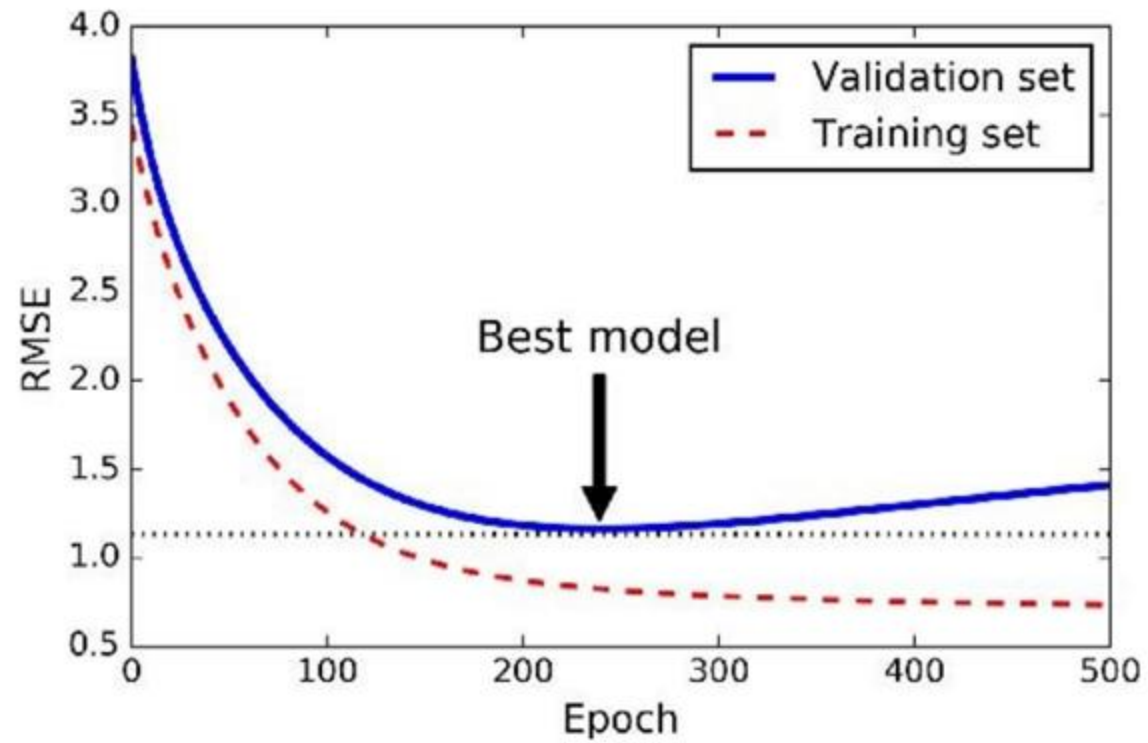
- Linear model results in under fitting with large bias
- Polynomial of order 300 results in a large variance



How to experiment on Model Complexity ?



Handling Overfitting – Way 3



Regularization

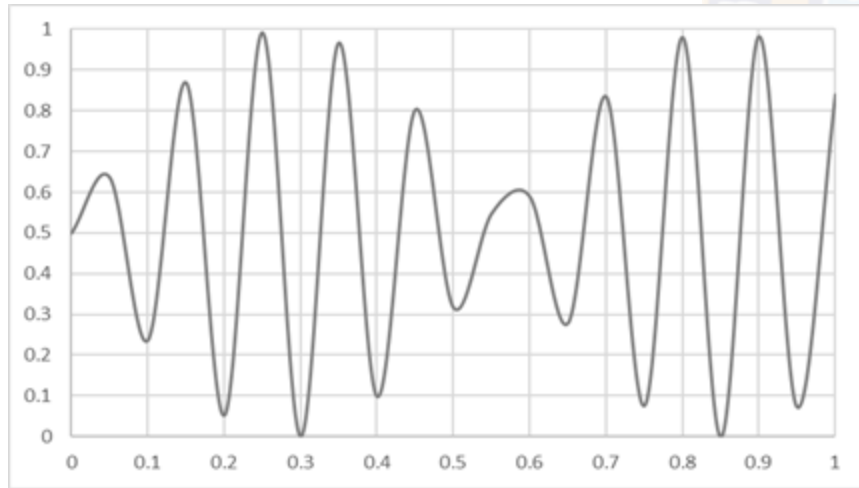
(*This notion is common for both Linear Regression – Module 3 and Logistic Regression – Module 4)



Overfitting vs Underfitting

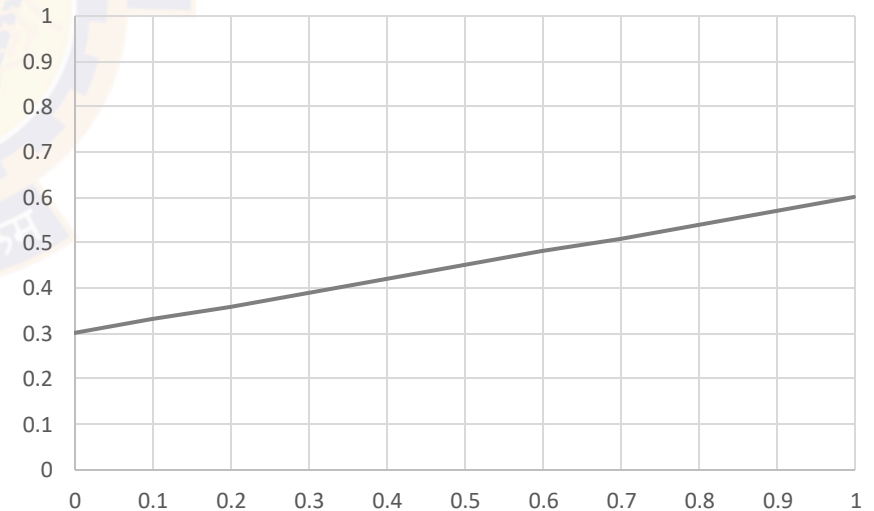
Overfitting

- Fitting the data too well
 - Features are noisy / uncorrelated to concept



Underfitting

- Learning too little of the true concept
 - Features don't capture concept
 - Too much bias in model



Regularization

- A method for automatically controlling the complexity of the learned hypothesis
- **Idea:** penalize for large values of θ_j
 - Can incorporate into the cost function
 - Works well when we have a lot of features, each that contributes a bit to predicting the label
- Can also address overfitting by eliminating features (either manually or via model selection)



Regularization

- Linear regression objective function

$$J(\theta) = \underbrace{\frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}_{\text{model fit to data}} + \underbrace{\frac{\lambda}{2} \sum_{j=1}^d \theta_j^2}_{\text{regularization}}$$

- λ is the regularization parameter ($\lambda \geq 0$)
- No regularization on θ_0 !



Understanding Regularization

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- Note that $\sum_{j=1}^d \theta_j^2 = \|\boldsymbol{\theta}_{1:d}\|_2^2$
 - This is the magnitude of the feature coefficient vector!

- We can also think of this as:

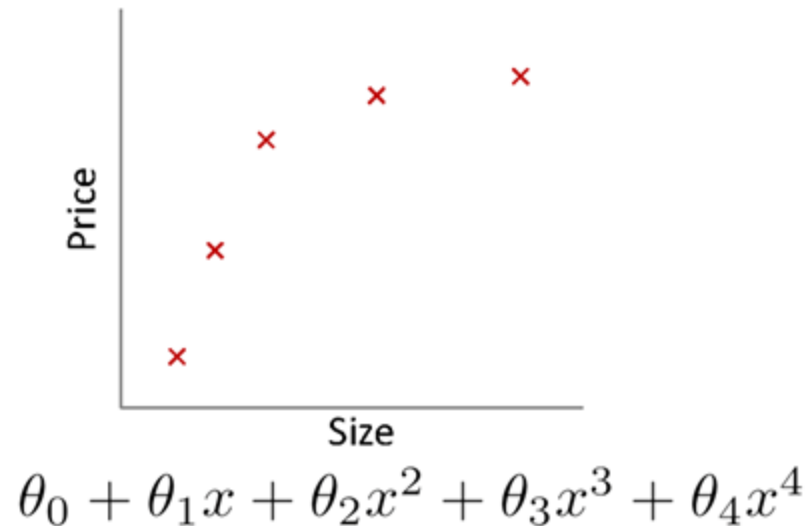
$$\sum_{j=1}^d (\theta_j - 0)^2 = \|\boldsymbol{\theta}_{1:d} - \vec{\mathbf{0}}\|_2^2$$

- L_2 regularization pulls coefficients toward 0

Understanding Regularization

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?

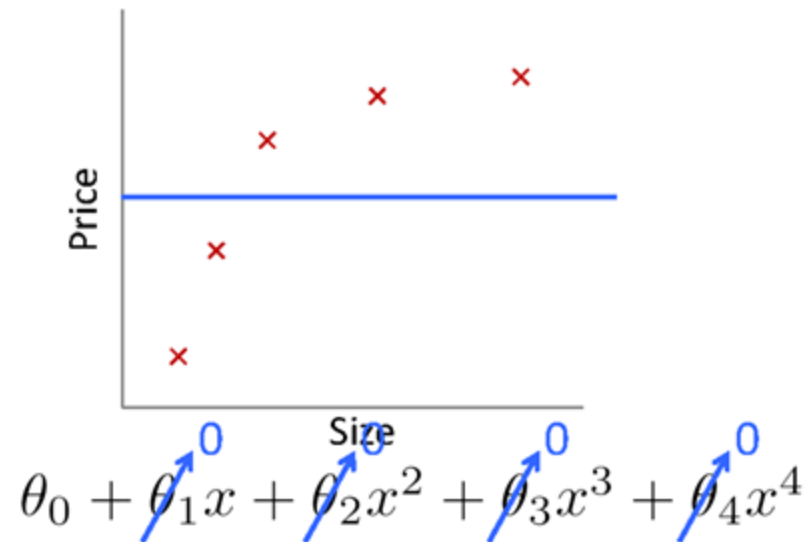


Based on example by Andrew Ng

Understanding Regularization

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(x^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?



Based on example by Andrew Ng

Regularization Ridge Regression / Tikhonov regularization

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- Fit by solving $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- Gradient update:

$$\frac{\partial}{\partial \theta_0} J(\boldsymbol{\theta})$$

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)$$

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta})$$

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} - \lambda \theta_j$$

regularization

- We can rewrite the gradient step as:

$$\theta_j \leftarrow \theta_j (1 - \alpha \lambda) - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Regularization

Lasso Regression (Least Absolute Shrinkage and Selection Operator Regression)

- Cost Function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^d |\theta_j|$$

- Fit by solving $\min_{\theta} J(\theta)$

- Gradient update:

$$\begin{aligned} \frac{\partial}{\partial \theta_0} J(\theta) \quad & \theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) \\ \frac{\partial}{\partial \theta_j} J(\theta) \quad & \theta_j \leftarrow \theta_j - \frac{\alpha}{n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) \mathbf{x}_j^{(i)} - \alpha \lambda \text{sign}(\theta_j) \end{aligned}$$

regularization

$$\text{where } \text{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

Regularization Elastic Net

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + r \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2 + \frac{(1-r)}{2} \lambda \sum_{j=1}^d |\theta_j|$$

Control the regularization using **the Mix Ratio “r”**:

When,

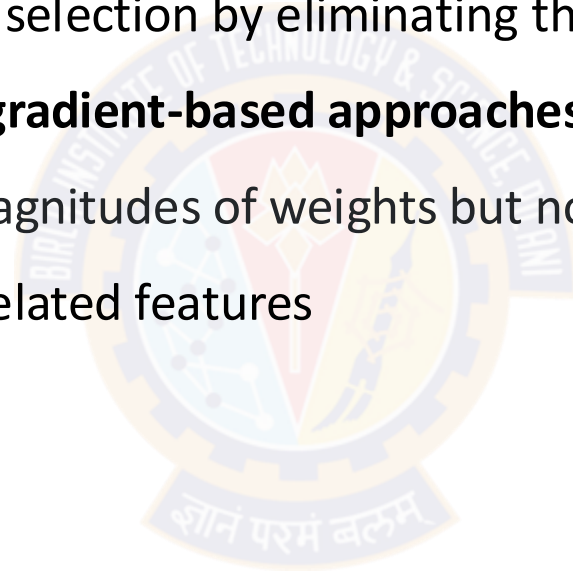
r = 0, Elastic Net is equivalent to Ridge Regression,

r = 1, it is equivalent to Lasso Regression

```
from sklearn.linear_model import ElasticNet  
ElasticNet(alpha=0.1, l1_ratio=0.5)
```

How to choose the right Regularization? Common Usage & Observation

- L1 regularization has the ability to set some coefficients to 0 exactly leading to a *sparse* model
- L1 regularization helps in feature selection by eliminating the features that are not important
- **L1 cannot be used efficiently in gradient-based approaches since it is not-differentiable unlike L2**
- L2 will in general lead to small magnitudes of weights but not exactly 0.
- Elastic net – Prefer in Highly correlated features



Numerical Exercise – For Student Practice

Fit a linear regression. Show only the first iteration of Gradient descent algorithm using learning rate of **0.02** for the following data , if the Relative Risk of Coronary Heart Disease is believed to be only linearly dependent on BMI as well as Diastolic Pressure. Assume the intercept of the regression model as **5** and the slope of independent variables as **-0.03 (negative)**.

Patient	Systolic Pressure mm Hg	Diastolic Pressure mm Hg	BMI	Waist Circumference Threshold cm	RR-CHD (Relative Risk of Coronary Heart Disease)
1	140	80	35	100	1.81
2	120	80	25	80	1.22
3	130	100	30	60	1.71

Apply a regularization on the same problem with regularization constant 5 and apply GD for 2 iterations & interpret the results. Try both ridge regression as well as lasso regression. Below equation is changed only for ridge regression. Students must apply appropriate GD update equation changes for this problem.

Steps :

1. Identification of the equations $y = w_0 + w_1X_1 + w_2X_2$
2. Cost function & derivative
 1. $W_0' = w_0 - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y))$
 2. $W_1' = w_1 * (1 - \text{learning rate} * \text{regularization constant}) - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y) * x_1)$
 3. $W_2' = w_2 * (1 - \text{learning rate} * \text{regularization constant}) - 1/3 * \text{learning rate} * (\text{sum}(w_0 + w_1X_1 + w_2X_2 - y) * x_2)$
3. Apply the equations

Answer:

Model	w_0	w_1	w_2
Linear	5.0016	0.0477	0.1793
Ridge	5.0016	0.0507	0.1823
Lasso	5.0016	0.1477	0.2793

Interpretation

- ◆ **Without Regularization**

Weights increase significantly.

- ◆ **Ridge**

Weights shrink slightly due to multiplication by 0.9.

- ◆ **Lasso**

Adds fixed penalty → can push weights toward zero faster.

Python Lab Exercise – For Student Practice

Go to your virtual lab file under : Machine Learning ☐ LabCapsule 2 Linear _
Polynomial Regression ☐ 3 Polynomial Regression

Download ML_Lab 5 PolynomialRegression.ipynb

Try to change the degree to {2, 5, 7, 10} in the below function line :
`PolynomialFeatures(degree=3)`

Observe and interpret on the performance of the training data

Note: Training vs Test Data split is not coded in this implementation. Below are the suggestions to experiment further:

- Add few hundreds of data (Use synthetic data generation python libraries available : Refer here)
- Split the data into 80% train vs 20% test set
- Built the polynomial model using above four different degree on training set
- For each of the model apply in the test set
- Find the MSE for both training data and separately for test data
- Plot the four experiment results in a plot and interpret the notion of overfit vs underfit
- Suggest which degree is a best fit .



Thank you